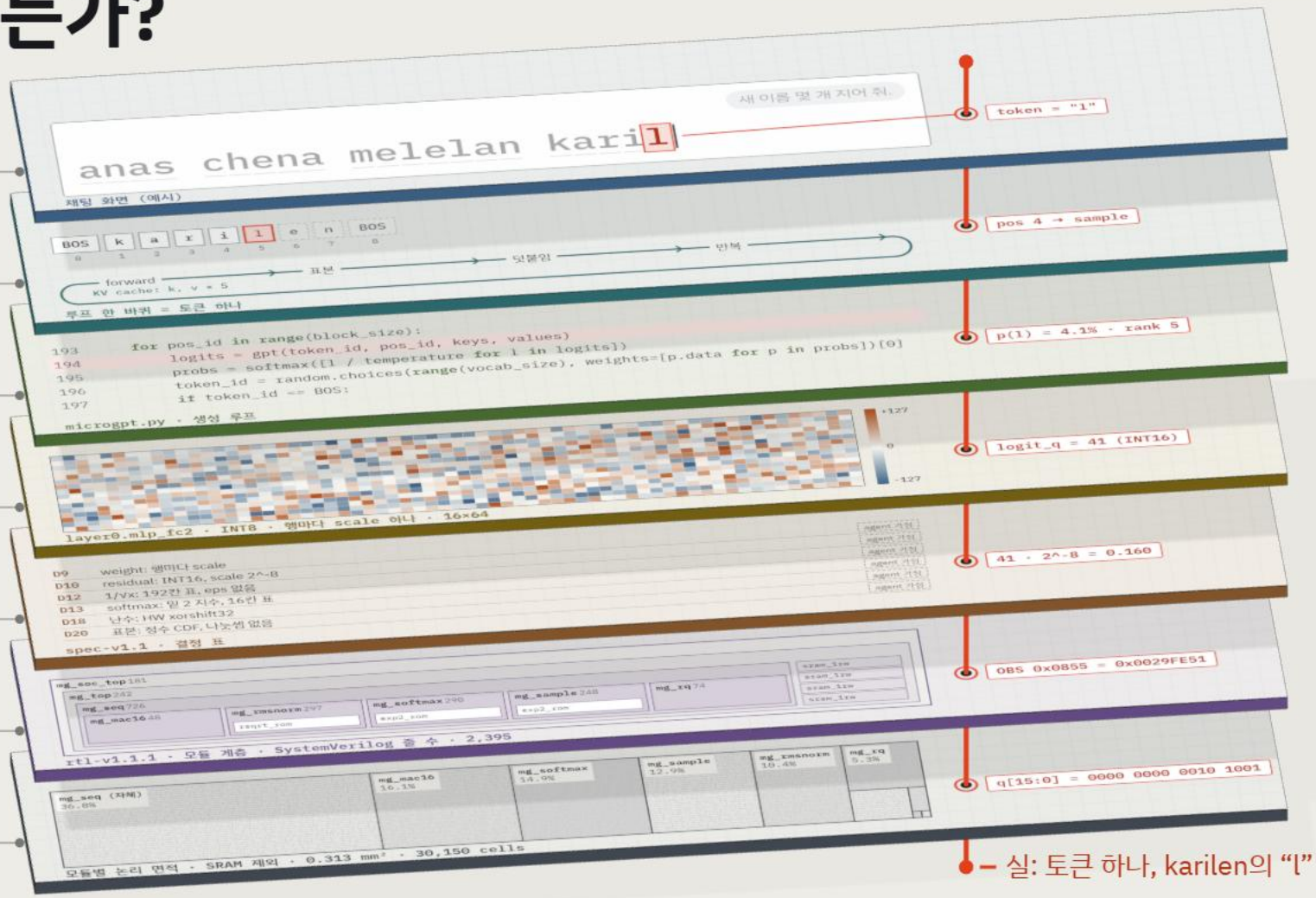
tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

토큰 하나는 어디서 오는가?

- 서비스
- 루프
- Python
- 정수 연산
- Spec
- RTL
- 셀

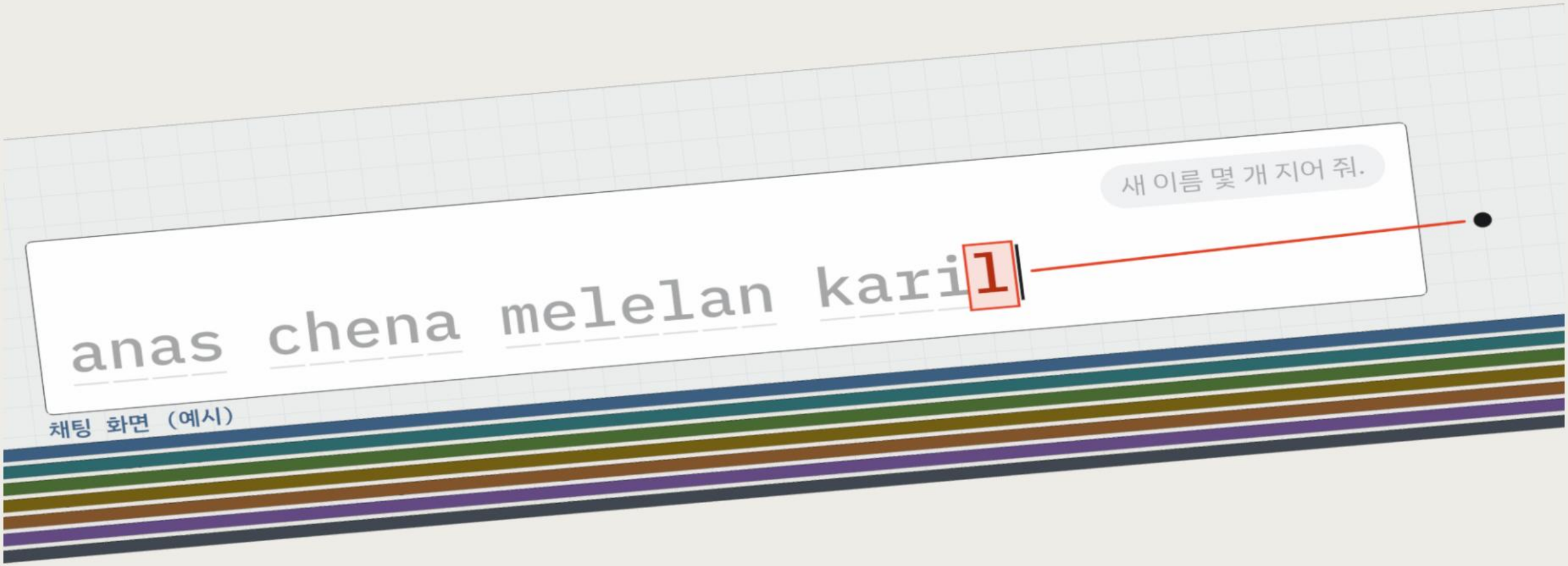


토큰 하나를 채팅 서비스에서 표준 셀까지 따라가고, AI agent 팀 셋이 그 하드웨어를 어떻게 만들었는지 본다.

채팅은 토큰 하나씩 답한다

토큰 1개 = 글자 1개

연출 화면 · 이름은 RTL 시뮬레이션 출력



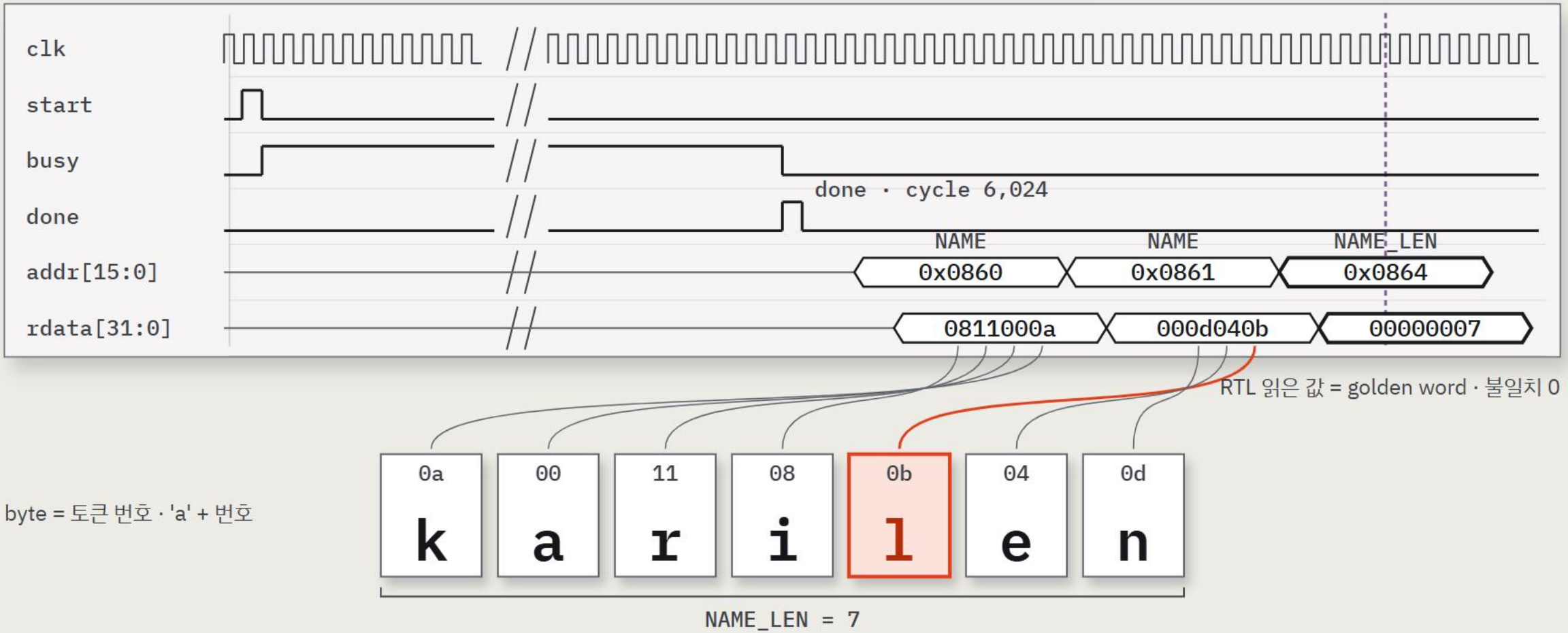
토큰 하나에서 멈추고 그것이 어디서 왔는지 묻는다. 채팅 창은 연출이고, 그 안의 이름은 RTL이 실제로 낸 것이다.

출처 names: RTL simulation log (facts names)

이 이름은 GPU가 아니라 RTL 시뮬레이션에서 나왔다

RTL 시뮬레이션 `mg_top · icarus · arch-golden-v1.0 · s2a-n03`

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성



GPU 아님 · 로직 블록 하나

agent 팀 셋 **사양 · 설계 · 검증**

< 20시간

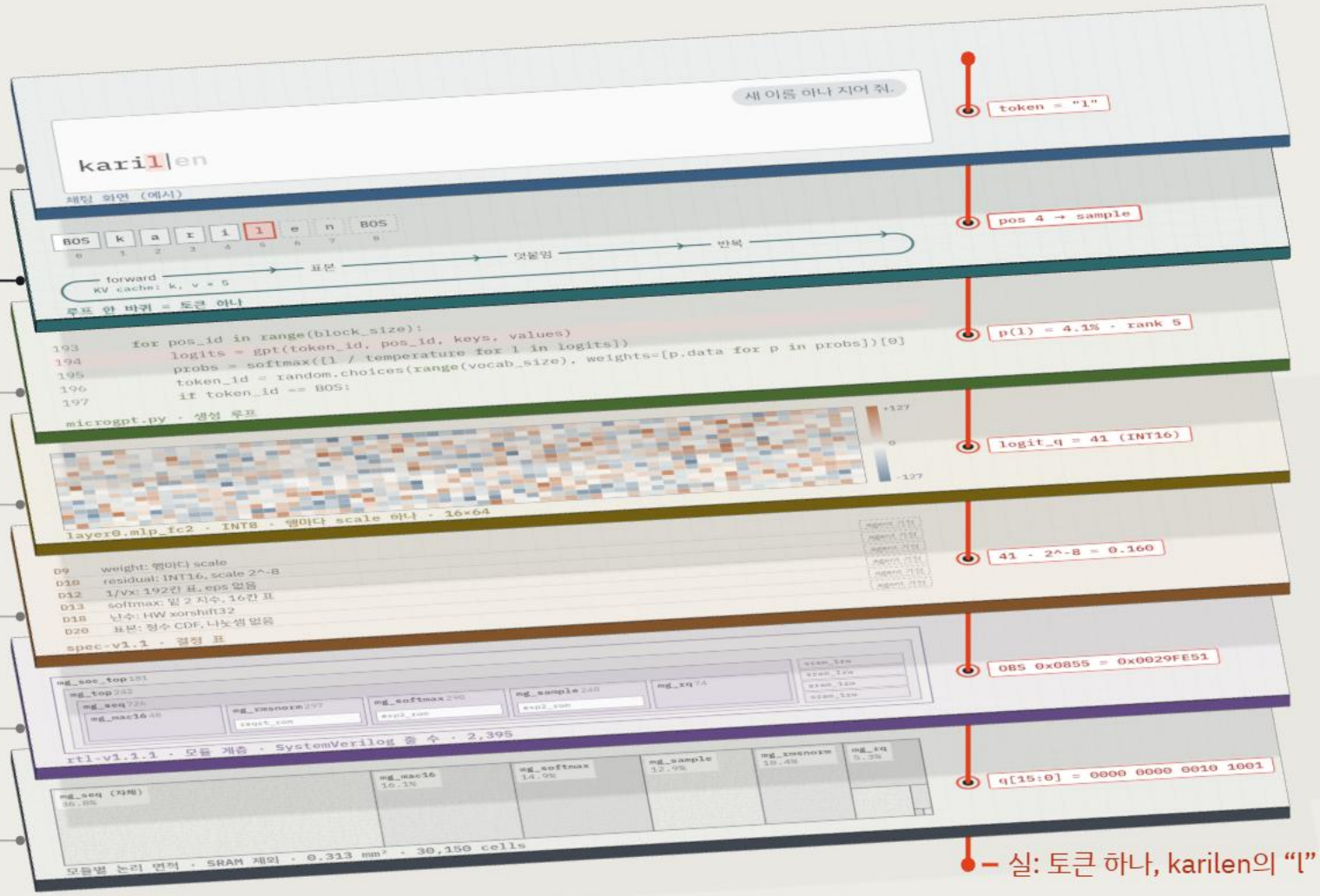
karilen은 설계의 관측 memory에서 읽어 낸 것이다. RTL 시뮬레이션과 sky130 합성까지만 했고 tape-out은 하지 않았다.

출처 RTL test log NAME / NAME_LEN words

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

거의 모든 LLM 서비스 뒤에는 루프 하나가 있다

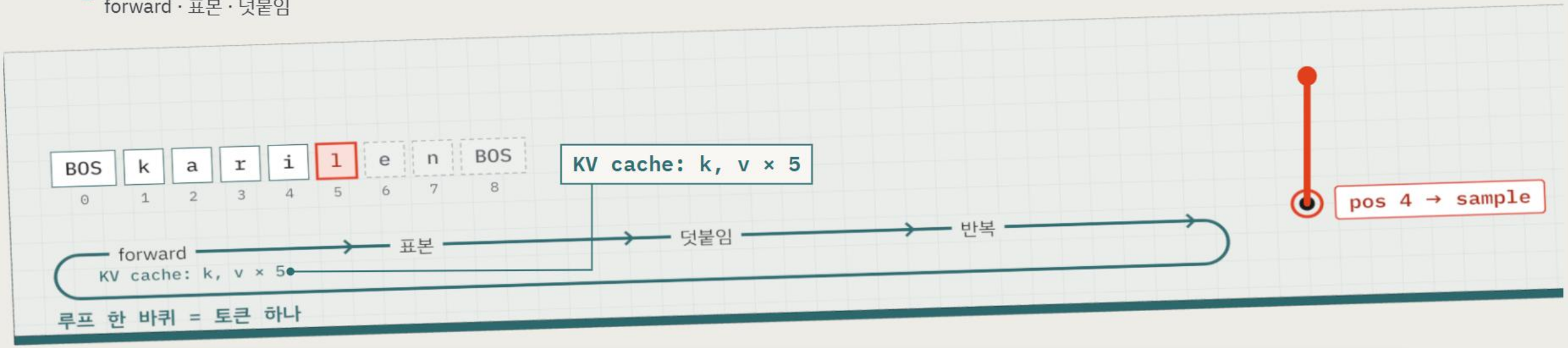
- 서비스
- **루프**
  forward · 표본 · 덧붙임
- Python
- 정수 연산
- Spec
- RTL
- 셀



루프가 한 번 돌면 토큰이 딱 하나 나온다

루프

forward · 표본 · 덧붙임



microgpt.py · 191–199

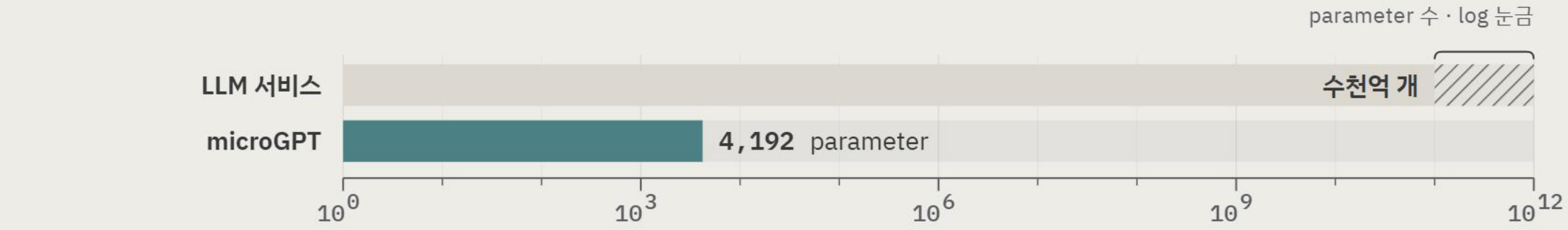
```
토큰화      191  token_id = BOS
            192  sample = []
            193  for pos_id in range(block_size):
forward     194      logits = gpt(token_id, pos_id, keys, values)
확률        195      probs = softmax([l / temperature for l in logits])
표본        196      token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
멈춤        197      if token_id == BOS:
            198          break
덧붙임      199      sample.append(uchars[token_id])                    "kari" + "l"
```

토큰으로 바꾸고, forward를 돌리고, 다음 토큰마다 확률을 얻고, 하나를 뽑아 덧붙이고, 다시 돈다.

출처 microgpt.py generate loop

같은 루프가 파라미터 4,192개에서도 돈다

루프
forward · 표본 · 덧붙임



1	16	4	27	16
n_layer	n_embd	n_head	26 + BOS	block_size
층	채널	attention head	토큰	문맥 위치
microgpt.py:75	microgpt.py:76	microgpt.py:78	microgpt.py:26	microgpt.py:77

1,000 학습에 쓴 이름 — microgpt.py:152 — **32,033** 목록의 이름



목록에 없는 새 이름

chena
melelan
karilen
janan

층 1, 채널 16, head 4, 토큰 27. 이름 32,033개 중 1,000개로 학습했다. 상용 모델도 수천억 규모에서 같은 루프를 돈다.

출처 facts.json model

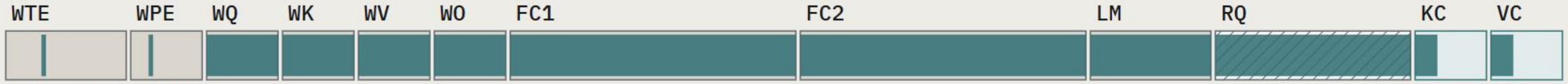
새 토큰마다 weight 전부와 자라는 cache를 다시 읽는다

루프

forward · 표본 · 덧붙임



spec-v1.1 · M · weight + KV cache



WTE · WPE: 한 행

5 / 16

토큰 하나에 읽는 양 (위치 4) · 읽는 byte 하나에 곱셈 약 하나



위치마다 key와 value를 남기고, 뒤 위치는 읽기만 한다. 그래서 생성은 대개 memory 문제다.

출처 spec section M: KV row layout

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

기준 모델은 부동소수점 Python 파일 하나다

서비스

루프

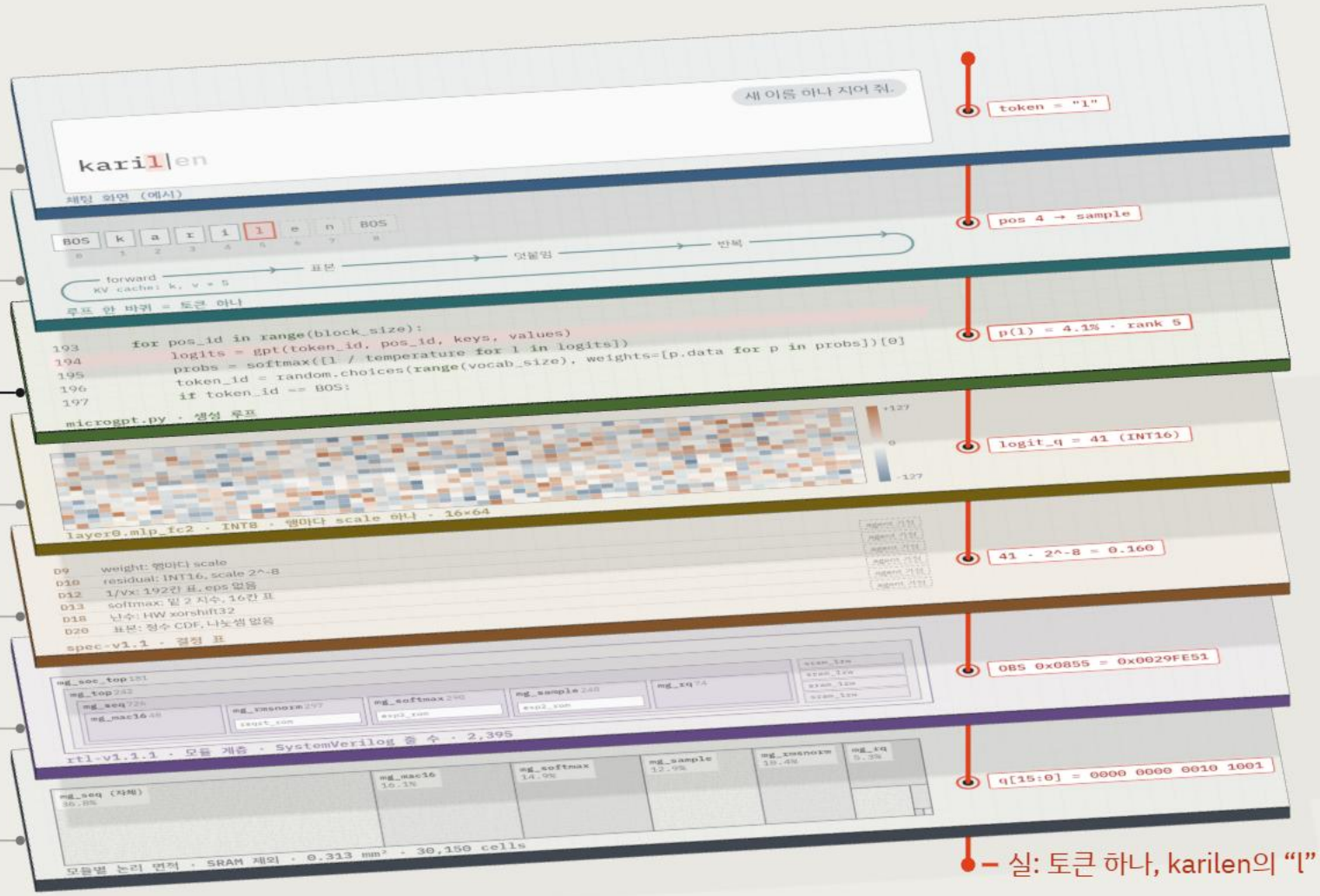
Python
199줄, 부동소수점

정수 연산

Spec

RTL

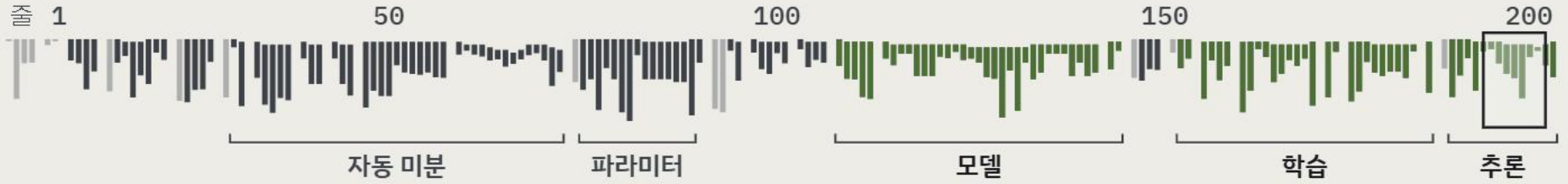
셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · Yosys 검증한 쪽 **verif (독립)** 사람의 설계 결정 □ **0**

학습까지 포함한 GPT 전체가 평범한 파일 하나다

microgpt.py · 파일 하나 · 순수 Python · float



- 서비스
- 루프
- **Python**
- 정수 연산
- Spec
- RTL
- 셀

```
190      keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
191      token_id = BOS
192      sample = []
193      for pos_id in range(block_size):
194          logits = gpt(token_id, pos_id, keys, values)
195          probs = softmax([l / temperature for l in logits])
196          token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
197          if token_id == BOS:
```

microgpt.py · 생성 루프

출처 **Karpathy · microgpt.py · gist 8627fe009c40f57531cb18360106ce95**

라이브러리 없는 순수 Python 약 200줄. 뒤의 모든 층은 이 파일의 번역이다.

출처 Karpathy, microgpt.py

위치 하나에 곱셈이 최대 4,016번 든다

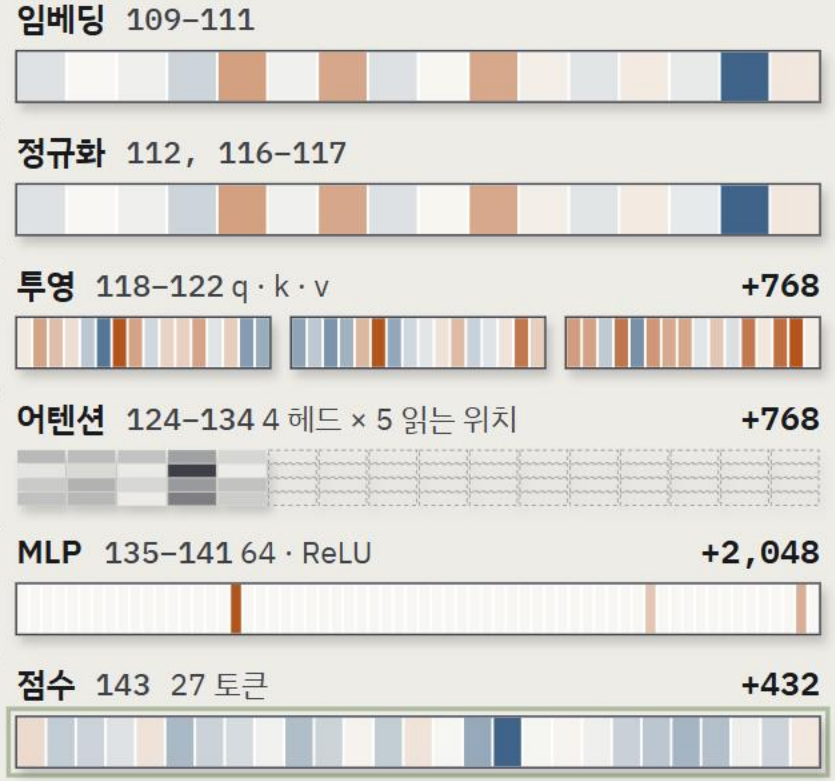
forward 한 위치 · gpt() · microgpt.py 108–144

```
123         x_attn = []
124         for h in range(n_head):
125             hs = h * head_dim
126             q_h = q[hs:hs+head_dim]
127             k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
128             v_h = [vi[hs:hs+head_dim] for vi in values[li]]
129             attn_logits = [sum(q_h[j] * k_h[t][j] for j in rang…
130             attn_weights = softmax(attn_logits)
131             head_out = [sum(attn_weights[t] * v_h[t][j] for t i…
132             x_attn.extend(head_out)
133         x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
134         x = [a + b for a, b in zip(x, x_residual)]
135         # 2) MLP block
136         x_residual = x
137         x = rmsnorm(x)
138         x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
139         x = [xi.relu() for xi in x]
140         x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
141         x = [a + b for a, b in zip(x, x_residual)]
142
143     logits = linear(x, state_dict['lm_head'])
144     return logits
```

위치당 곱셈 수

4,016

3504 + 32(p+1)
최대 · p = 15



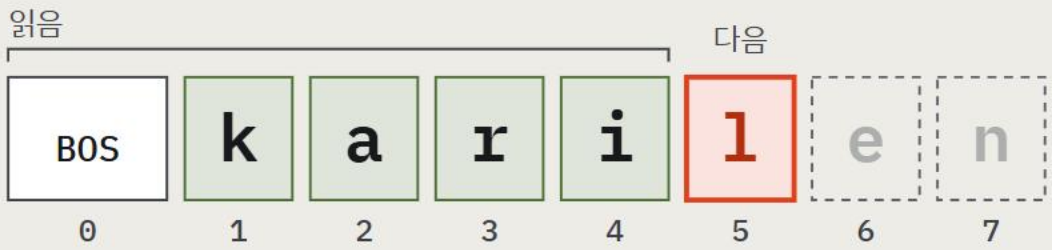
도구 **lab/instrumented.py · 118 × gpt()**

임베딩, 정규화, q · k · v 투영, attention, 작은 MLP, 토큰 27개 점수.

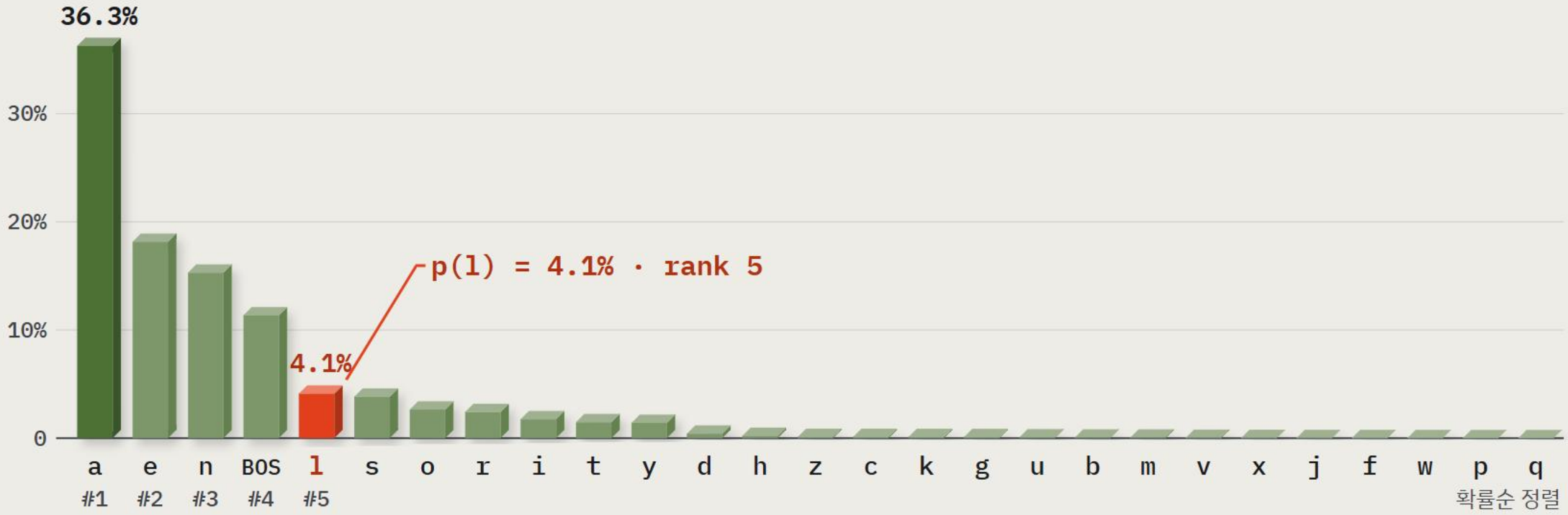
출처 facts.json model (instrumented run, lab/instrumented.py)

우리 글자는 5위, 확률은 4.1%였다

다음 글자 확률 · float 모델 · softmax(logits / T) · T = 0.5



karilen



k, a, r, i 다음 1등은 a다. 우리 글자 l은 5위라, 표본 뽑기가 실제로 일을 한다.

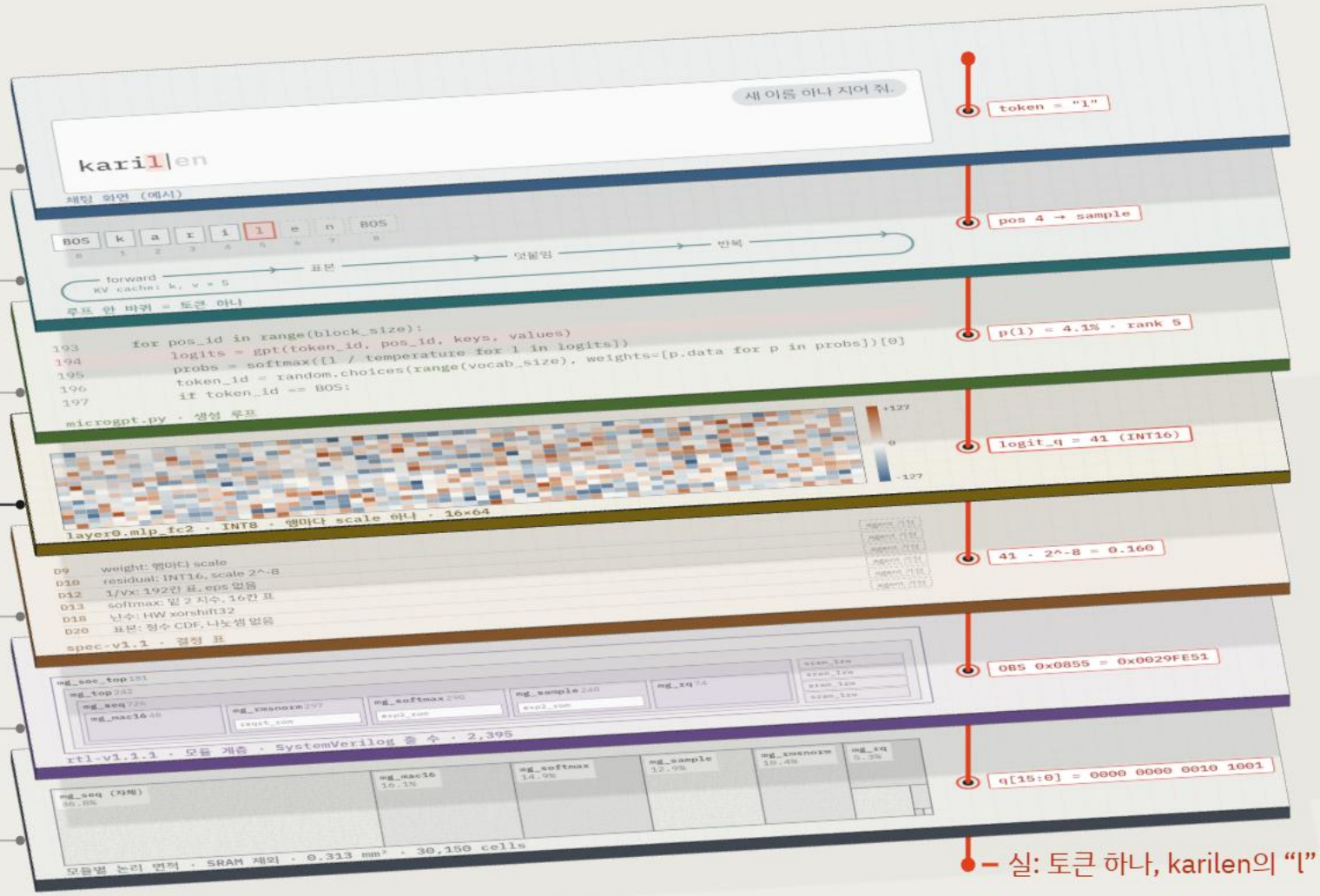
출처 `facts.json thread (float)`

3막 · 정수 연산

실리콘은 정수를 원하니, 모든 수를 정수로 바꿨다

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

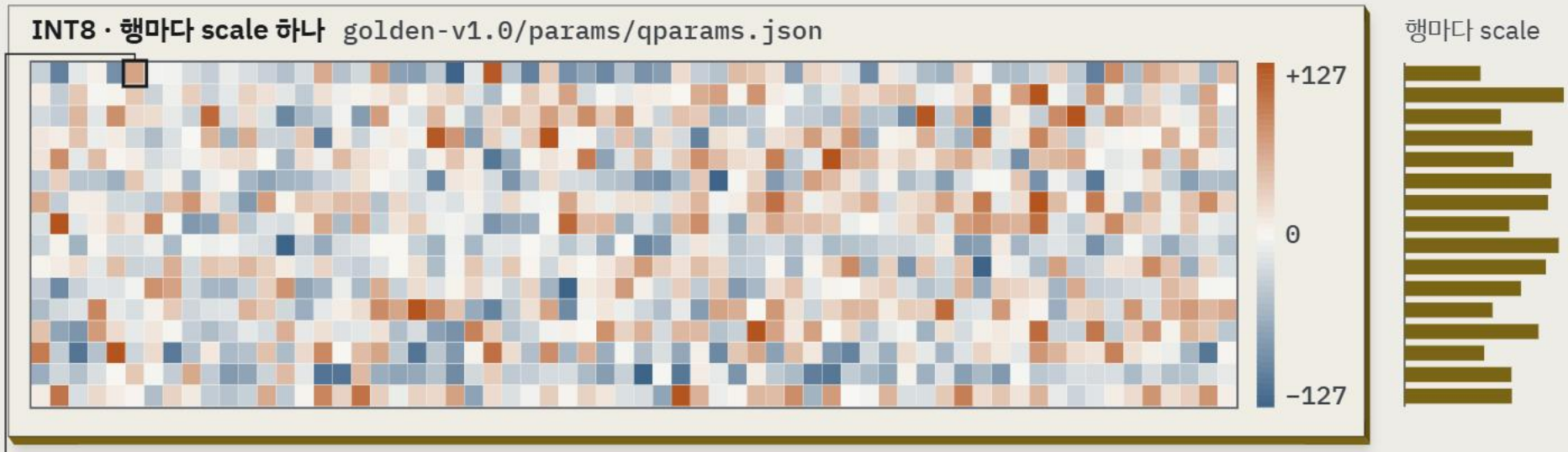
- 서비스
- 루프
- Python
- **정수 연산**
  INT8 weight, 고정 scale
- Spec
- RTL
- 셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · Yosys 검증한 쪽 **verif (독립)** 사람의 설계 결정 □ **0**

weight는 행마다 scale 하나를 둔 8비트가 됐다

weight · layer0.mlp_fc2 · 16 × 64



서비스
루프
Python
정수 연산
Spec
RTL
셀

residual 흐름 · INT16 · × 2^-8 thread 위치

−159	167	−132	−279	507	49	502	−61	53	312	22	−23	−23	33	−736	230

만든 쪽 **arch agent** 도구 `lab/m2_lab.py`

실제 layer0.mlp_fc2 행렬의 양자화 전후. residual stream은 16비트가 됐다.

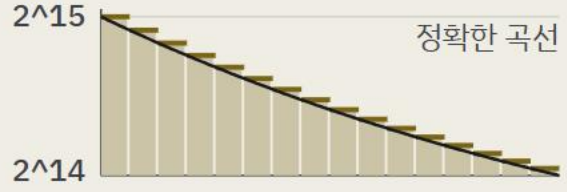
출처 `weights.json, golden qparams (lab/m2_lab.py)`

어려운 함수는 16칸 · 192칸 표가 됐다

어려운 함수 → 표 · golden-v1.0/luts

지수 함수 softmax · sampler

`round(2^15 · 2^(−f/16))`



f 0	8000	7a93
f 2	7560	7066
f 4	6ba2	6712
f 6	62b4	5e84
f 8	5a82	56ac
f 10	52ff	4f7b
f 12	4c1c	48e2
f 14	45cb	42d5

`luts/exp2_f4.hex` 칸 **16**

1 / 제곱근 RMSNorm

`round(2^18 / √m) · m = 64 … 255`



64	8000	7f03	7e0c	7d1a	7c2e	7b46	7a64	7987	78ae	77da	770a	763e	7576	74b2	73f2	7336
80	727d	71c7	7115	7066	6fba	6f12	6e6c	6dc9	6d29	6c8b	6bf0	6b58	6ac2	6a2f	699e	690f
96	6883	67f9	6771	66ea	6666	65e4	6564	64e6	6469	63ef	6376	62fe	6289	6215	61a2	6132
112	60c2	6054	5fe8	5f7d	5f13	5eab	5e44	5ddf	5d7a	5d17	5cb5	5c55	5bf5	5b97	5b3a	5ade
128	5a82	5a28	59d0	5978	5921	58cb	5876	5822	57cf	577c	572b	56db	568b	563d	55ef	55a2
144	5555	550a	54bf	5475	542c	53e4	539c	5355	530f	52c9	5284	5240	51fc	51b9	5177	5135
160	50f4	50b4	5074	5035	4ff6	4fb8	4f7a	4f3d	4f01	4ec5	4e8a	4e4f	4e14	4dda	4da1	4d68
176	4d30	4cf8	4cc1	4c8a	4c53	4c1d	4be7	4bb2	4b7e	4b49	4b15	4ae2	4aaf	4a7c	4a4a	4a18
192	49e7	49b6	4985	4955	4925	48f5	48c6	4897	4868	483a	480c	47df	47b2	4785	4758	472c
208	4700	46d5	46aa	467f	4654	462a	4600	45d6	45ad	4583	455b	4532	450a	44e2	44ba	4492
224	446b	4444	441e	43f7	43d1	43ab	4385	4360	433b	4316	42f1	42cc	42a8	4284	4260	423d
240	4219	41f6	41d3	41b1	418e	416c	414a	4128	4106	40e5	40c3	40a2	4082	4061	4040	4020

`luts/rsqrt8.hex` 칸 **192**

softmax의 지수는 2진 표, 제곱근의 역수는 192칸 표다. 값은 golden ROM 파일에서.

출처 `golden exp2_lut, rsqrt_lut`

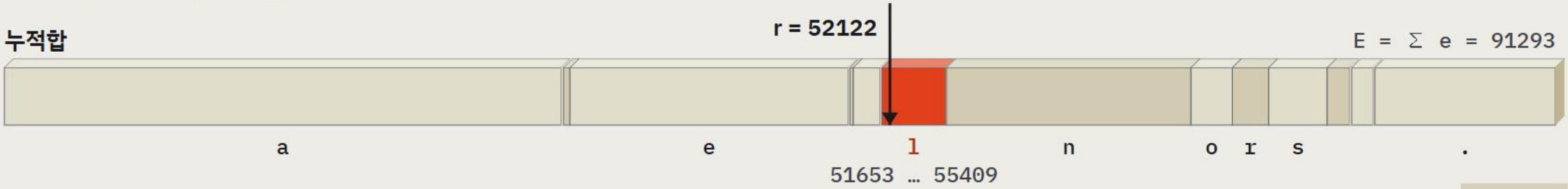
주사위까지 정수라서 이름이 똑같이 나올 수 있다

주사위 · sample_q · xorshift32 · seed 42



`r = (x · E) >> 32 = (2452169066 · 91293) >> 32 = 52122`

sampler **나눗셈 없음**



토큰마다 한 번

RTL = golden · 44 이름 · **불일치 0**



32비트 xorshift와 정수 누적합이 토큰을 고르고, sampler에는 나눗셈이 없다. 설계와 golden 모델이 같은 수를 같은 순서로 쓴다.

출처 `facts.json thread (integer sampling)`

정수로 바꿔도 정확도는 거의 잃지 않았다

비용 · float → 정수

held-out loss 변화 2,000 이름

+0.0019%

NLL 2.3786 → 2.3786

0.0019% 예산 3%

float와 같은 1위 토큰

99.54%

float · Python



a e n . l

logit **0.158** 확률 **4.1%** 순위 **5**

정수 · INT16 logit_q = 41 (INT16)



a e n . l

logit **41 · 2^-8 = 0.160** 확률 **4.1%** 순위 **5**

사람의 설계 결정 □ 0 spec 결정: agent 가정, 시험함

따로 둔 이름 2,000개에서 loss는 +0.0019% 움직였고, 1등 토큰은 99.54% 일치했다.

출처 facts.json integer, thread

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

그 선택은 모두 spec 한 문서에 적혀 있다

서비스

루프

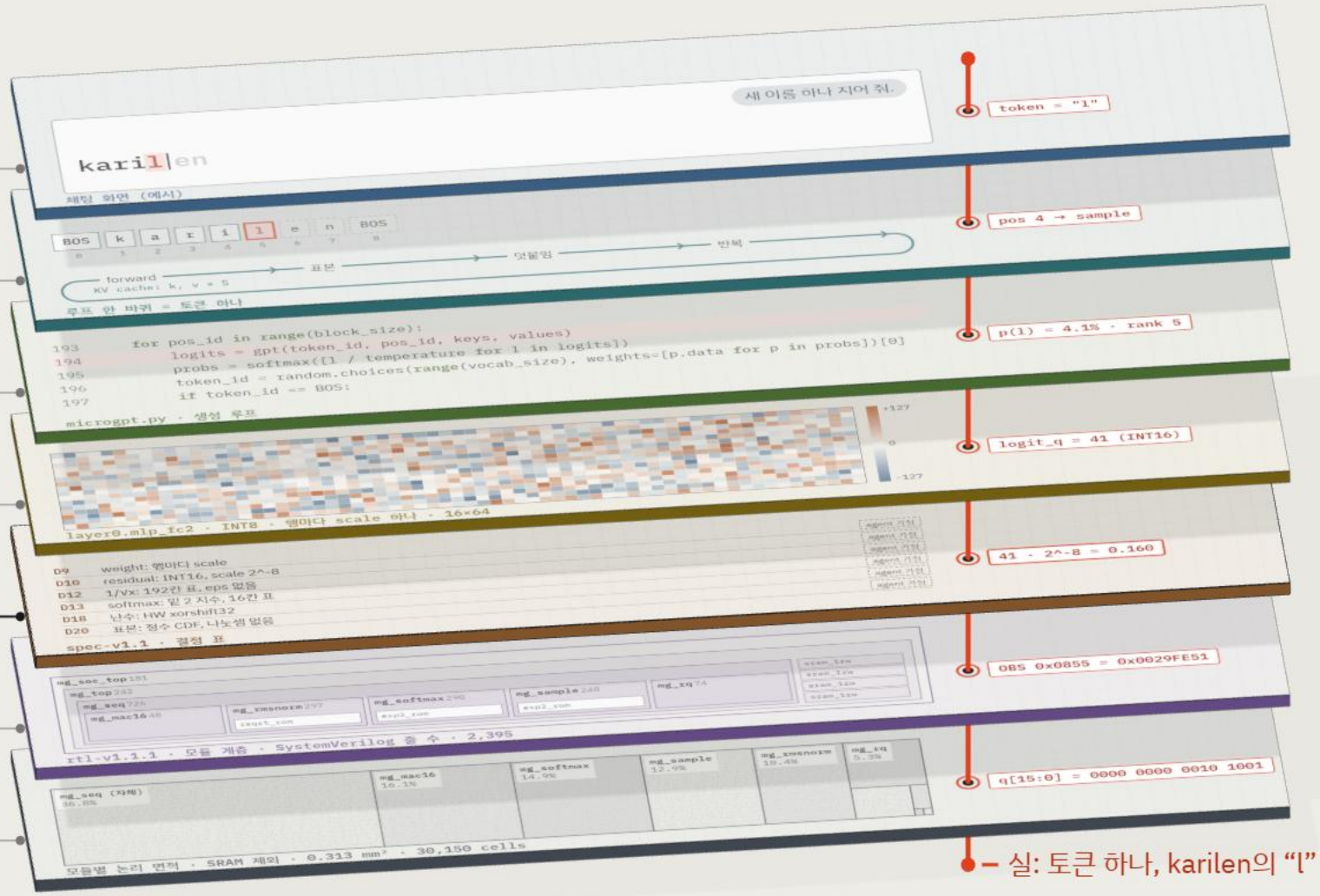
Python

정수 연산

Spec
두 팀이 함께 따르는 계약

RTL

셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · Yosys 검증한 쪽 **verif (독립)** 사람의 설계 결정 **0**

결정마다 근거와 버린 대안을 함께 적었다

spec-v1.1/SPEC.md · 396 줄 · D 결정 표 :358

spec 원문 | 모든 행: agent 가정

F 기능 | N 수치 | I 경계 인터페이스 | M 메모리 지도 | T 타이밍 계약 | O 관측점 | S 생성 규약 | P 성능 목표 | G golden API | D 결정 표 | Q 열린 모호함

- 서비스
- 루프
- Python
- 정수 연산
- **Spec**
- RTL
- 셀

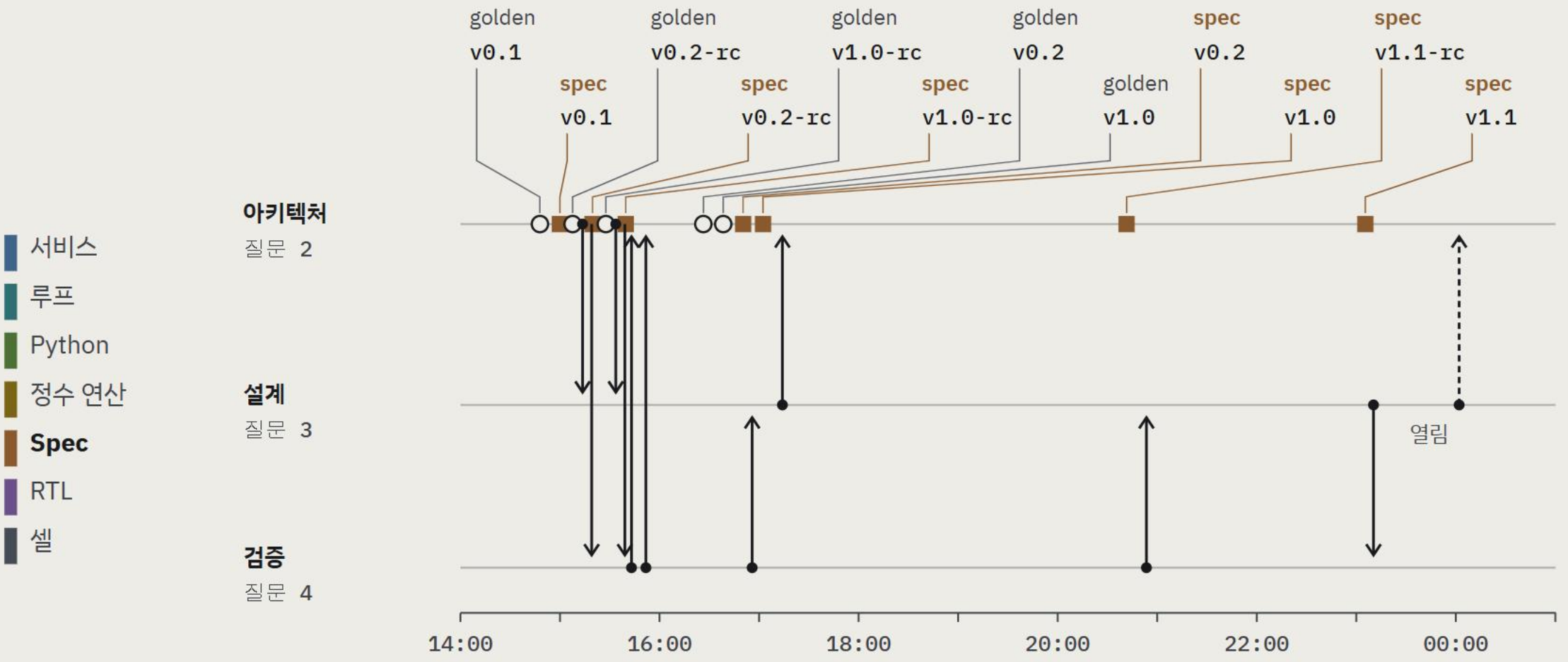
#	결정	근거	버린 대안
D9	**weight는 행마다 scale(per-row), requantize m · s도 행마다**	per-tensor는 NLL은 같지만(−0.02%) float 분포와의 KL이 2.6배(5.4e−5 → 1.4e−4), argmax 일치 99.45% → 99.09%	per-tensor
D10	**residual은 INT16, scale 2^−8**	INT8(2^−5)은 KL 1.5배(8.3e−5), 그리고 lm_head 입력 absmax 4.19가 INT8 2^−5 범위(±3.97)를 넘는다	INT8 residual
D12	**RMSNorm은 eps를 빼고, 짝수 shift 정규화 + 192칸 역제곱근 표**	768칸 표(가수 10비트)와 NLL 같음(+0.00%), KL 6.1e−5 vs 5.4e−5. 표가 1/4	768칸 표, 정확한 정수 제곱근 + 나눗셈(회로가 크다)
D13	**softmax는 밑 2 지수(소수 4비트, 16칸 표) + 역수 한 번(정확한 나눗셈) + 확률 UINT8**	소수 8비트(256칸) · 6비트(64칸)와 NLL · KL이 같다(5.35e−5). 점수가 작아(최대 |c·z|가 작음) 거친 표로 충분하다	256칸 · 64칸 표, 원소마다 나눗셈(head마다 T번)
D18	**난수는 HW xorshift32 하나, SEED 레지스터, 토큰마다 한 번**	golden과 testbench가 같은 정수 난수열을 같은 순서로 소비해야 한다(M3 bit-exact 규약). 생성기를 HW에 두면 testbench가 난수를 공급할 handshake가 없다. xorshift32는 XOR 셋과 shift 셋뿐이다	testbench가 포트로 난수 공급(위치마다 handshake), LFSR 한 비트씩(32 cycle)
D20	**정수 CDF: r = (x · E) >> 32, cum > r인 첫 토큰**	나눗셈 · 나머지 없이 [0, E)의 균등에 가까운 r을 만든다(치우침 ≤ E/2^32 < 2^−12)	r = x mod E(나눗셈), 확률을 정규화한 뒤 CDF(나눗셈 27번)

기능, 수 형식, 포트, memory map, 타이밍, golden 모델 API. 설계와 검증이 함께 시작할 수 있다.

출처 spec-v1.1 결정 표 (SPEC.md 원문)

spec · golden이 12판 나왔고, 답이 나온 spec 질문은 새 판이 됐다

00_shared/handoff/arch · 아키텍처 판과 질문



도구 `release.py · questions/`

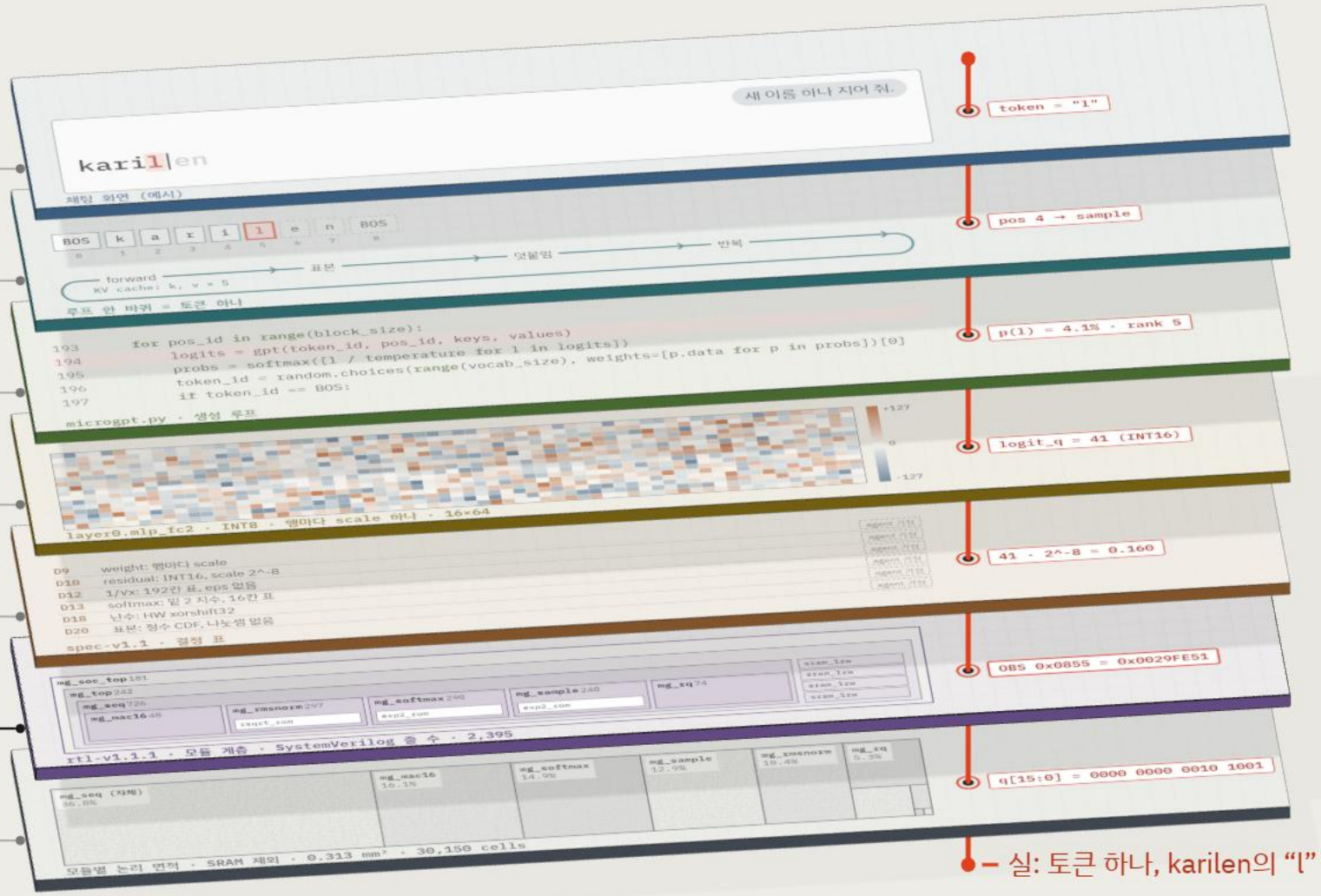
불분명한 곳이 있으면 팀이 질문을 올렸고, 그 답이 새 spec 판이 됐다.

출처 `facts.json spec, process.questions (release.py, questions/)`

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

spec은 SystemVerilog 모듈이 됐다

- 서비스
- 루프
- Python
- 정수 연산
- Spec
- **RTL**
  SystemVerilog 모듈
- 셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · Yosys 검증한 쪽 **verif (독립)** 사람의 설계 결정 □ **0**

설계 팀은 파일 11개에 2,395줄을 썼다

rtl-v1.1.1 · 모듈 계층

서비스
루프
Python
정수 연산
Spec
RTL
셀

2,395 SystemVerilog 줄 · 11 파일 — `rtl/*.sv`

mg_soc_top bus 감쌈	181
mg_top top	242
mg_seq sequencer	726
mg_mac16 곱셈 lane	48
mg_rmsnorm RMS norm	297
mg_rsqrt_rom 조회 표	214
mg_softmax softmax	290
mg_exp2_rom 조회 표	36
mg_sample 표본	248
mg_exp2_rom 조회 표	36
mg_rq requantize	74
mg_sram_1rw ×4 메모리	39

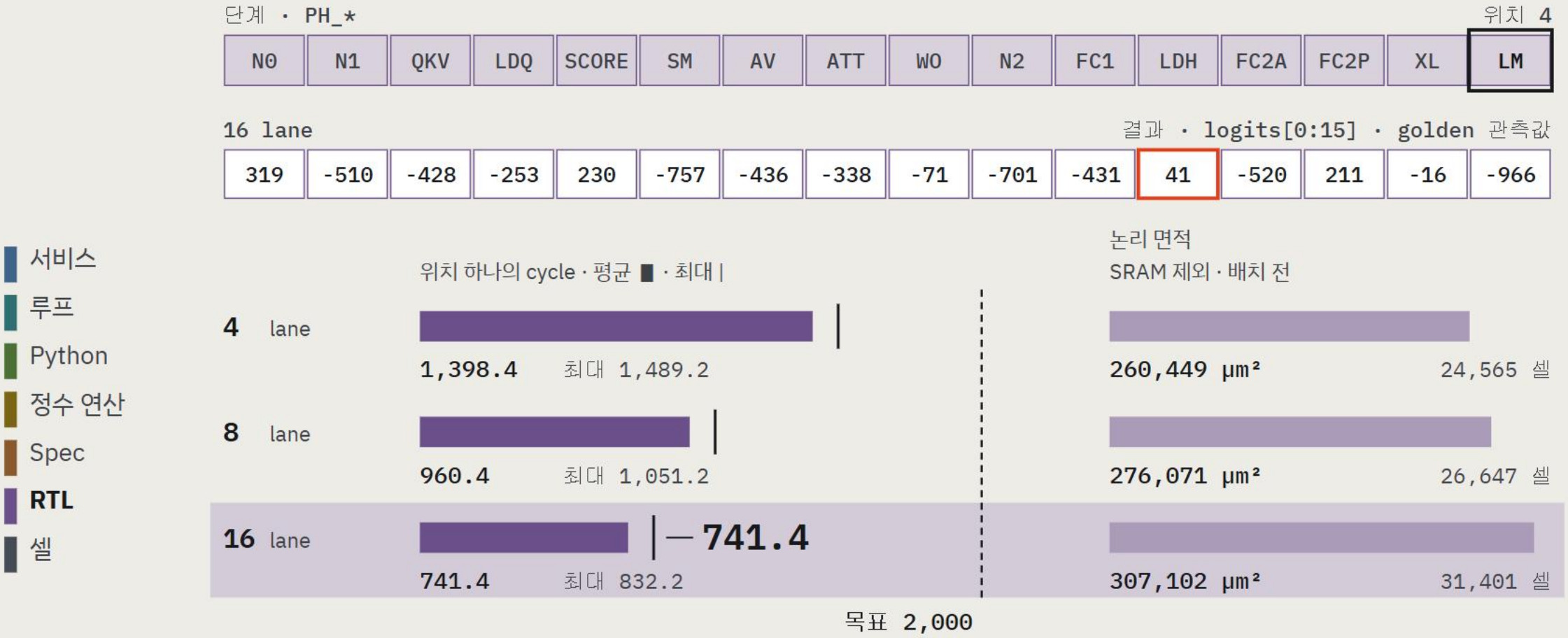
만든 쪽 **design agent** 도구 `gate.py · port_check.py`

bus wrapper, top, sequencer, 그리고 그것이 부리는 블록: 곱셈 lane, RMS norm, softmax, sampler, requantizer, 표 둘, memory.

출처 `facts.json rtl (gate.py, port_check.py)`

lane 16개면 위치당 평균 741 cycle

mg_seq.sv · phase_e · sequencer가 한 위치를 돈다



도구 dse.py

sequencer가 위치 하나를 단계별로 돈다. lane 4 · 8 · 16은 cycle과 면적을 맞바꾼다. 목표는 2,000 cycle이었다.

출처 facts.json rtl DSE (dse.py)

우리 글자의 점수는 memory에 41로 들어간다

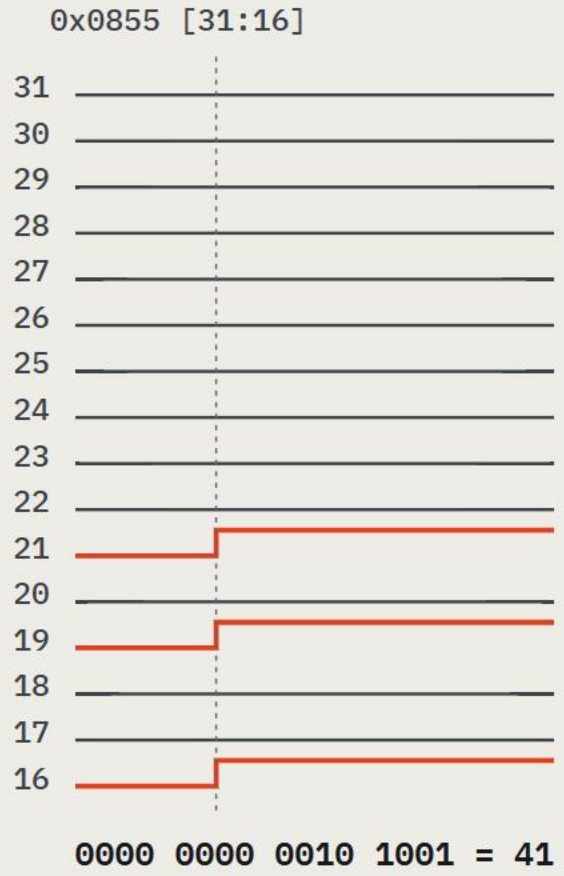
관측 메모리 · LOGITS · 0x0850 ~ 0x085D · INT16 × 27

golden packing으로 계산 · 파형 기록 아님

서비스
루프
Python
정수 연산
Spec
RTL
셀

word		[31:16]	[15:0]
0x0850	0xFE02013F	-510	319
0x0851	0xFF03FE54	-253	-428
0x0852	0xFD0B00E6	-757	230
0x0853	0xFEAEFE4C	-338	-436
0x0854	0xFD43FFB9	-701	-71
0x0855	0x0029FE51	41	-431
0x0856	0x00D3FDF8	211	-520
0x0857	0xFC3AFFF0	-966	-16
0x0858	0xFFE4F8F9	-28	-1799
0x0859	0xFFA60021	-90	33
0x085A	0xFDAEFE29	-594	-471
0x085B	0xFD6CFCDC	-660	-804
0x085C	0xFE5AFF9E	-422	-98
0x085D	0x000000B0	0	176

우리 글자 "l" · [11]



관측 memory word의 16비트. golden packing으로 그렸고, 파형 캡처가 아니다.

출처 facts.json thread hex, spec section 0 addresses

6막 · 셀

합성은 RTL을 표준 셀로 바꾼다

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

서비스

루프

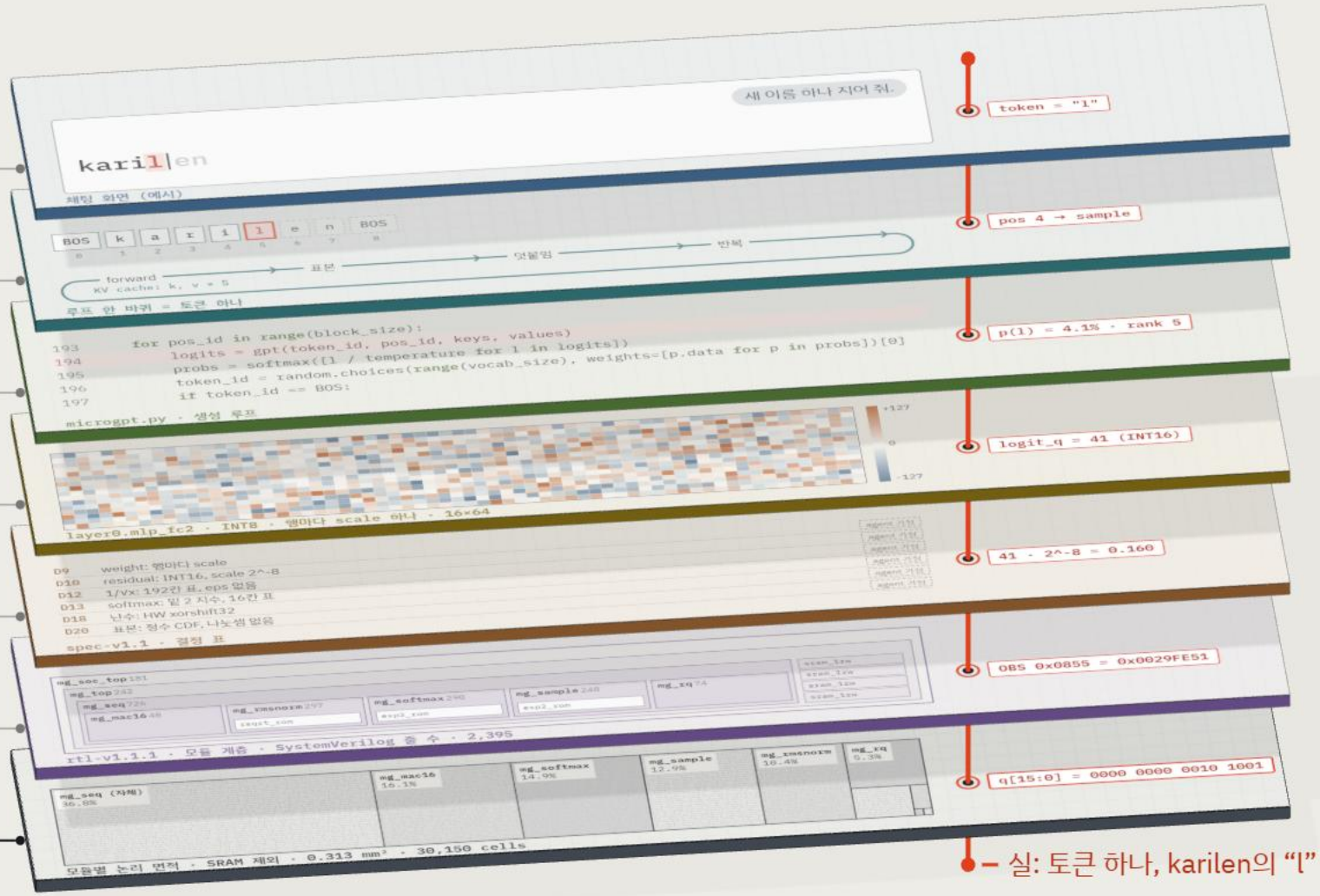
Python

정수 연산

Spec

RTL

셀
sky130 표준 셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · Yosys 검증한 쪽 **verif (독립)** 사람의 설계 결정 **0**

곱셈 lane은 논리의 16%뿐이다

SRAM 제외: mg_sram_1rw 4개는 blackbox로 뺌
논리만 · 합성 결과, 배치 · 배선 전

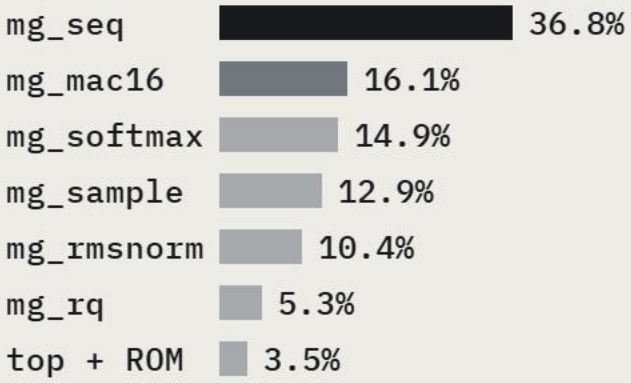
30,150
표준 셀

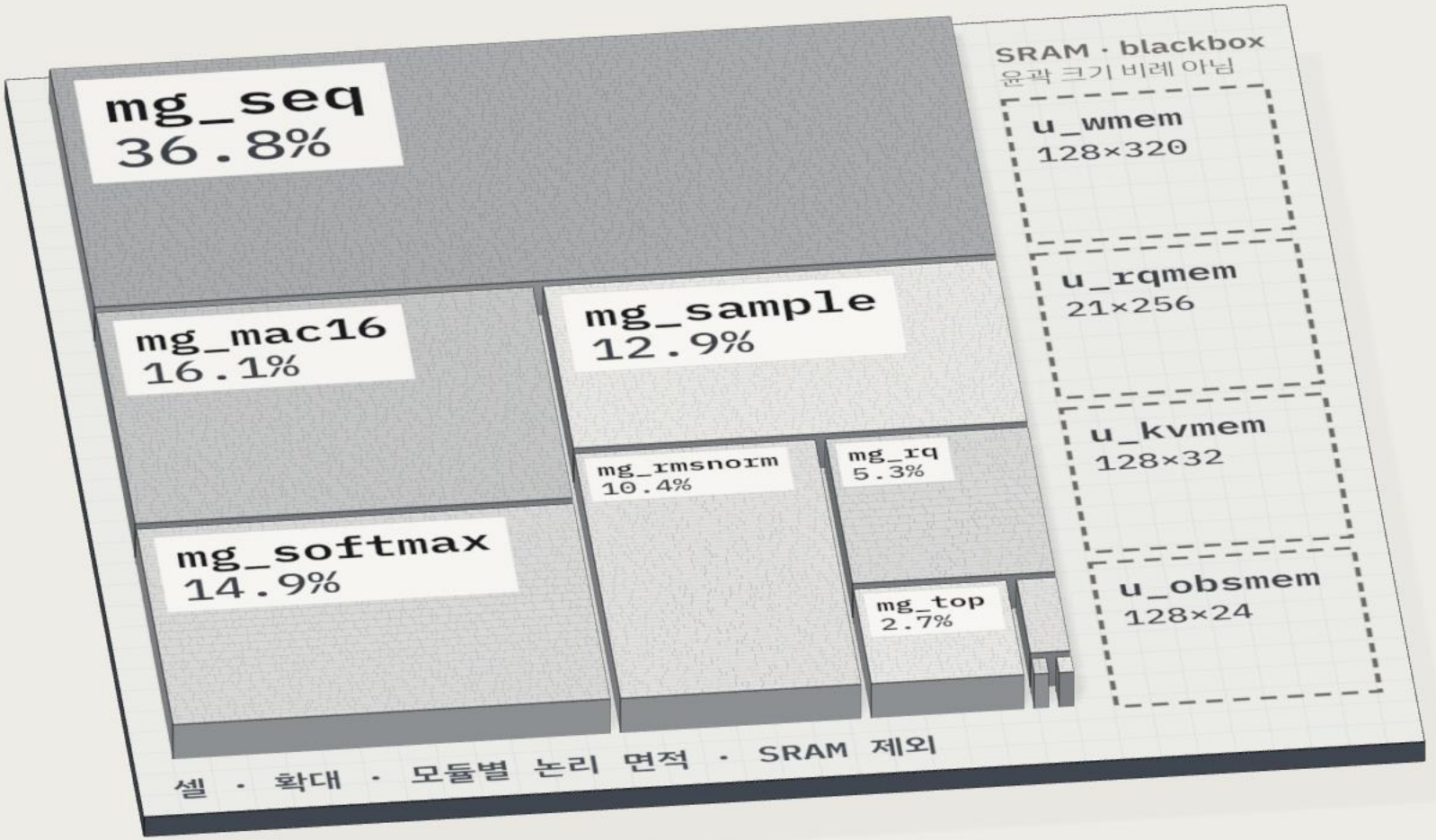
0.313 mm²
논리 면적

4,943
flip-flop

0
latch

논리 면적 중 비율 (모듈 자체 면적)

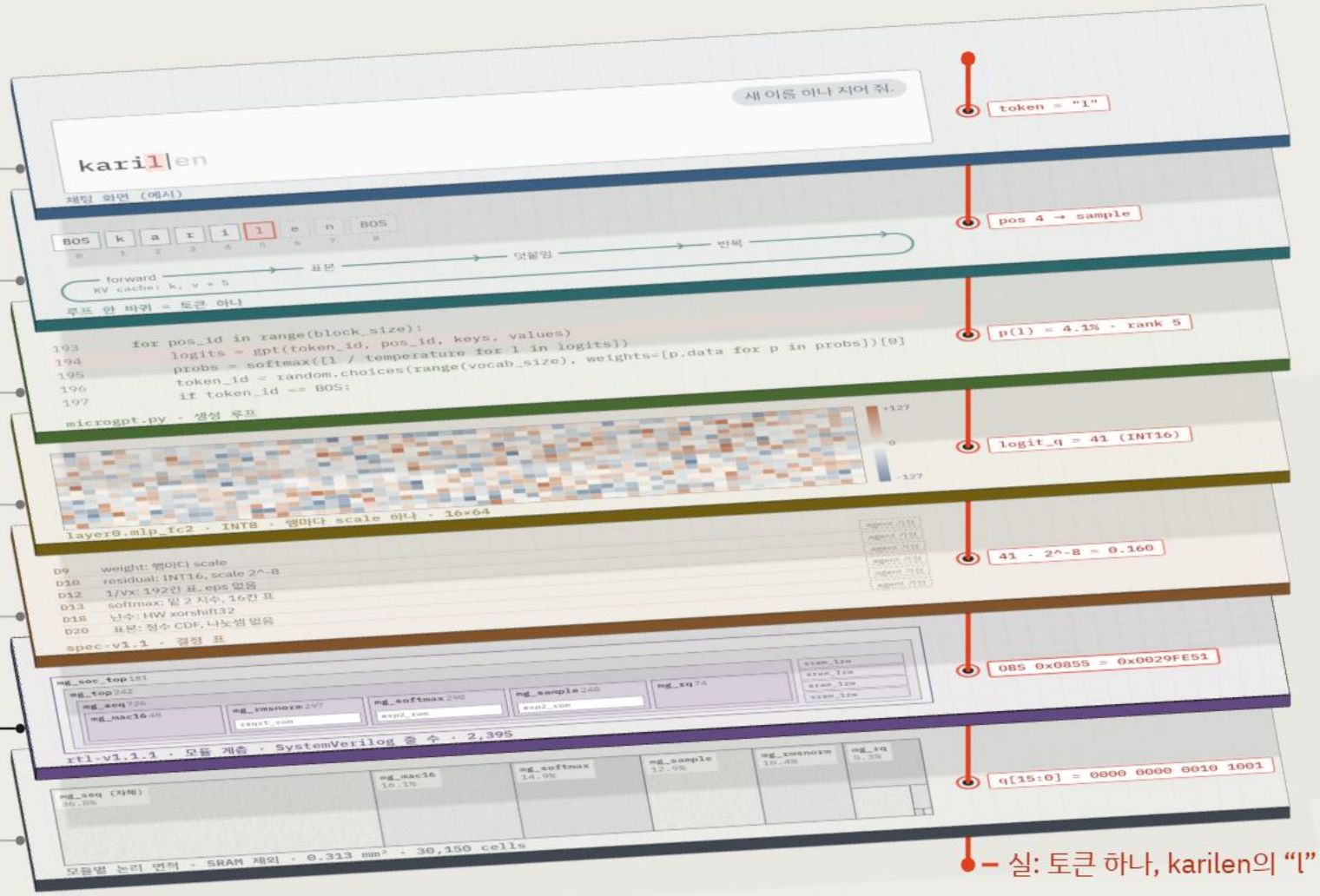




tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

검증은 설계를 spec에 대어 본다

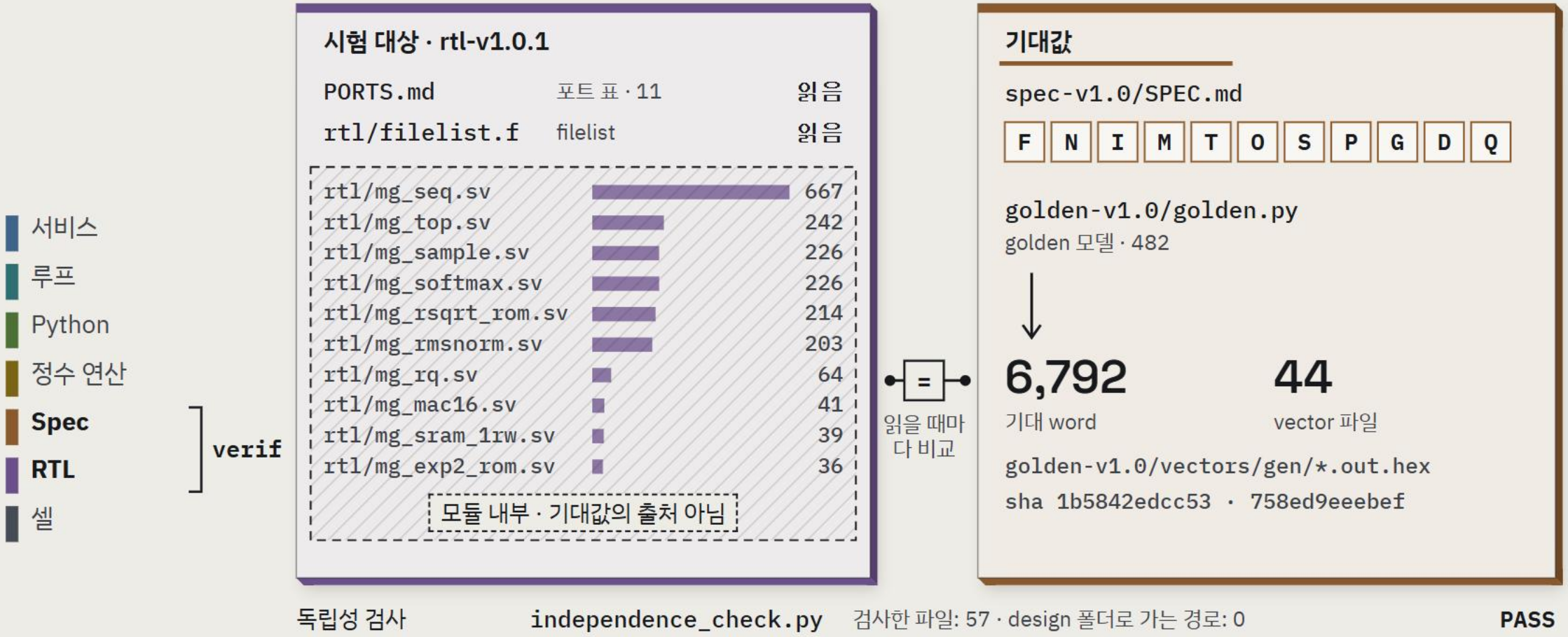
- 서비스
- 루프
- Python
- 정수 연산
- 검증
  - **Spec**
  - **RTL**
    SystemVerilog 모듈
- 셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · Yosys 검증한 쪽 **verif (독립)** 사람의 설계 결정 ◻ **0**

기대값은 모두 spec과 golden 모델에서 나왔다

검증 · 독립



만든 쪽 **verif agent (독립)** 도구 `independence_check.py`

검증은 정답을 정할 때 RTL 안을 보지 않았다. 포트 표와 file list는 봤다.

출처 `independence_check.py`

시뮬레이터 둘, 각각 5만 번 넘게 읽고 불일치 0

M3 회귀 시험 · 시뮬레이터 둘 · rtl-v1.0.1 · golden-v1.0



도구 `regress.py` · `cov_merge.py`

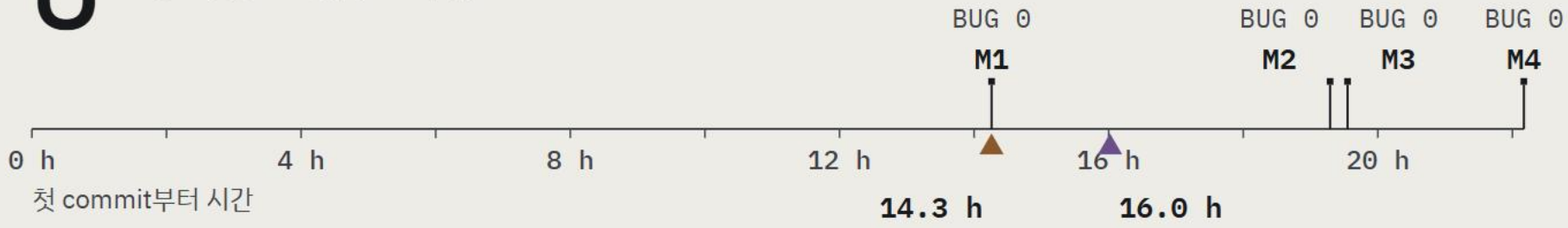
같은 시험을 서로 다른 시뮬레이터 둘에서 돌렸다. coverage bin 65개가 모두 켜졌다.

출처 `facts.json verif M3 (regress.py, cov_merge.py)`

이정표 검사는 RTL 결함을 못 찾았다. 잡은 건 더 앞이다

검증이 잡은 것

0 이정표 검증에서 찾은 RTL 결함



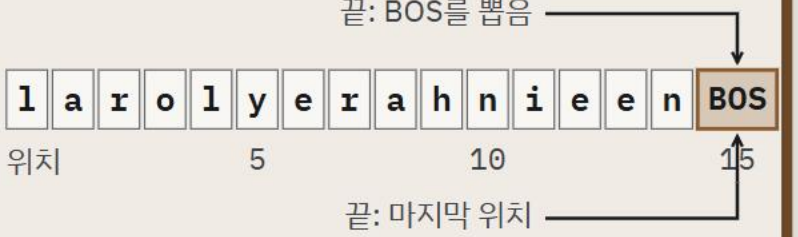
- 서비스
- 루프
- Python
- 정수 연산
- Spec
- **RTL**
- 셀

verif

spec rc 검토 · verif Spec · 14.3 h

`Q-verif-002 · spec-v1.0-rc`

이름 끝 경계 사례



있던 vector 길이: 0–8, 16

golden에 더함: stress15 · 15

design gate · rq smoke RTL · 16.0 h

`20_design/00_META/ledger.jsonl:18`

1비트 signed 상수

1'sd1 [1] = **−1**

sign

의도: +1

고친 줄 · rtl-v1.1.1 · mg_rq.sv:54

```
qi = (T_W + 1)'(q_q) + (T_W + 1)'(1);
```

spec 검토가 두 끝 조건이 겹치는 경계 사례를, design gate가 −1을 뜻하던 상수를 잡았다.

출처 `facts.json verif catches`

일부러 고장 낸 설계 76개 중 49개를 시험이 잡았다

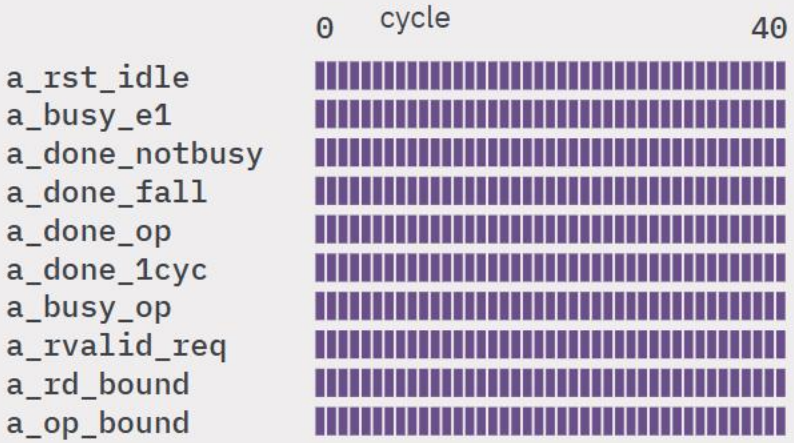
Formal · mutation

서비스
루프
Python
정수 연산
Spec
RTL
셀

verif

bounded formal · 포트 SVA

모든 입력 (solver)



M1 · rtl-v0.1.0	PASS · 40 · 13.1 s
M2 · rtl-v0.2.1	PASS · 40 · 643.0 s
M3 · rtl-v1.0.1	판정 안 함 · 메모리 부족

mutation · 첫 RTL · rtl-v0.1.0

49 / 76 변이 · icarus · 3.9 min



연산자

- **49** 시험이 잡음
- **17** formal 전용 줄 · simulation이 돌지 않음
- **10** 결과 변화 없음 추정 · 증명 전

+2 컴파일 안 됨

도구 **formal.py · mutate.py**

mutation은 첫 RTL에만 했고, 살아남은 것은 대개 formal 전용 줄이다. 깊이 40의 formal은 M1 · M2에서만 돌았다.

출처 facts.json verif formal, mutation (formal.py, mutate.py)

다시 위로: RTL과 golden의 이름이 비트까지 같다

다시 위로 · Service

- **서비스**
- 루프
- Python
- 정수 연산
- Spec
- RTL
- 셀

RTL simulation NAME · hex

seed 42 · 20 / 44

NAME	hex
anas	00 0d 00 12
chena	02 07 04 0d 00
melelan	0c 04 0b 04 0b 00 0d
karilen	0a 00 11 08 0b 04 0d
janan	09 00 0d 00 0d
caria	02 00 11 08 00
carila	02 00 11 08 0b 00
arana	00 11 00 0d 00
carinn	02 00 11 08 0d 0d
anna	00 0d 0d 00
arady	00 11 00 03 18
jaran	09 00 11 00 0d
alaran	00 0b 00 11 00 0d
jarie	09 00 11 08 04
ahrii	00 07 11 08 08
alari	00 0b 00 11 08
ratana	11 00 13 00 0d 00
lana	0b 00 0d 00
dian	03 08 00 0d
dosken	03 0e 12 0a 04 0d

bit 단위 일치



golden model NAME · hex

seed 42 · 20 / 44

NAME	hex
anas	00 0d 00 12
chena	02 07 04 0d 00
melelan	0c 04 0b 04 0b 00 0d
karilen	0a 00 11 08 0b 04 0d
janan	09 00 0d 00 0d
caria	02 00 11 08 00
carila	02 00 11 08 0b 00
arana	00 11 00 0d 00
carinn	02 00 11 08 0d 0d
anna	00 0d 0d 00
arady	00 11 00 03 18
jaran	09 00 11 00 0d
alaran	00 0b 00 11 00 0d
jarie	09 00 11 08 04
ahrii	00 07 11 08 08
alari	00 0b 00 11 08
ratana	11 00 13 00 0d 00
lana	0b 00 0d 00
dian	03 08 00 0d
dosken	03 0e 12 0a 04 0d

20 / 20 이름 일치

44 이름 · icarus · verilator · 불일치 0

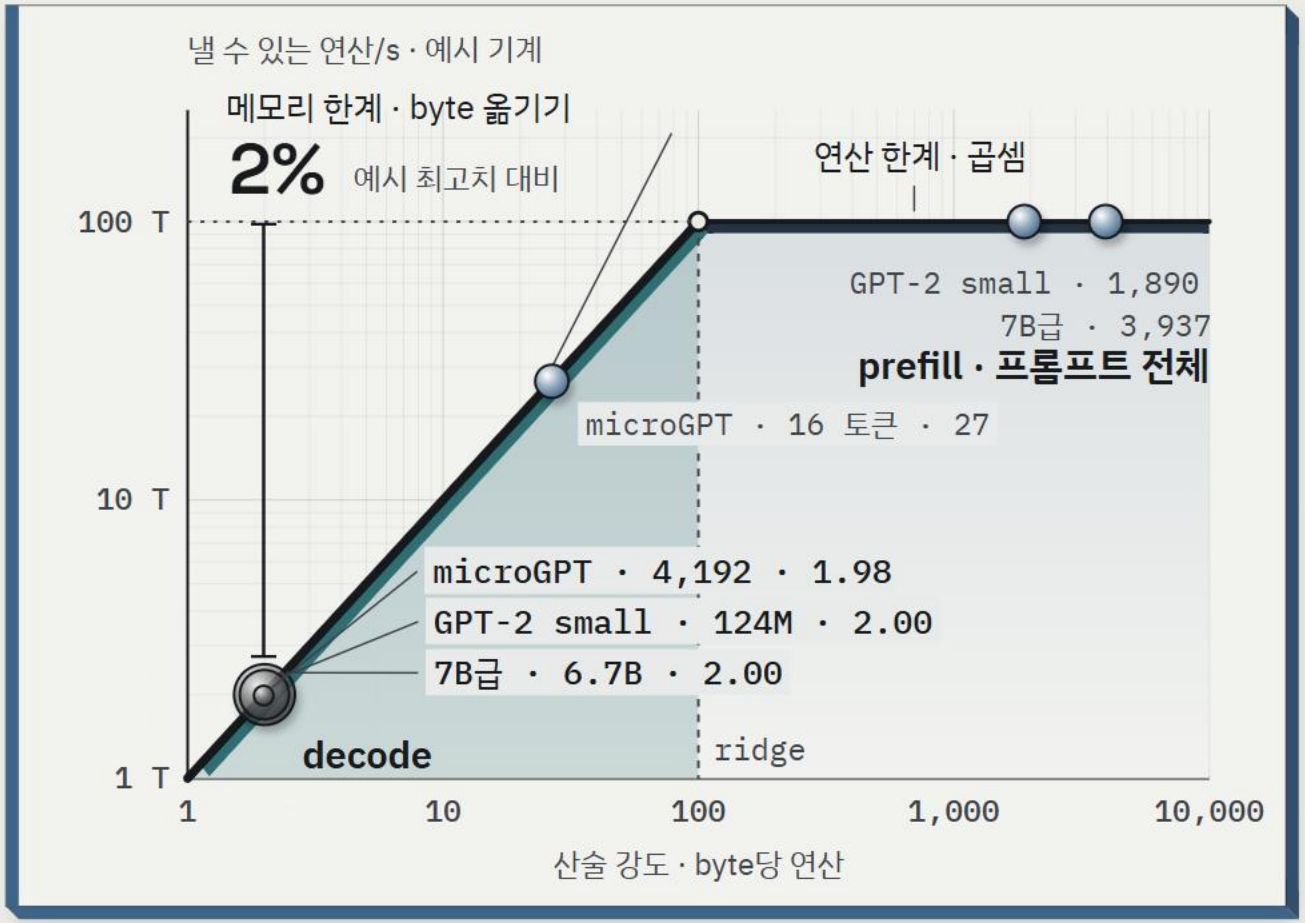
시뮬레이션 속 RTL과 golden 모델이 같은 seed에서 각각 이 이름들을 냈다.

출처 facts.json names

실제 규모에선 곱셈보다 데이터 이동이 생성을 막는다

실제 규모 · roofline

공개 정의로 계산, 측정 아님



decode · 한 번에 한 토큰 · batch 1

weight byte마다 곱셈-누적 한 번 → 2 연산 / byte

INT8 weight와 KV · 문맥 끝까지

모델	연산 / 토큰	byte / 토큰	연산 / byte
microGPT 4,192	8.0 K	4.0 KB	1.98
GPT-2 small 124M	285 M	143 MB	2.00
7B급 6.7B	14.3 G	7.1 GB	2.00

● weight ● KV cache

ridge · 예시 기계

100 TOPS ÷ 1 TB/s = 100 연산 / byte

계산 · 측정 아님 `build_backup.py`

토큰을 하나씩 쓰면, 토큰마다 weight와 key-value cache를 옮겨야 한다.

출처 `build_backup.py: computed from public definitions, not measured`

tape-out 안 함 · RTL 시뮬레이션 + sky130 합성

agent 팀 셋과 사람 하나가 만들었다

서비스

루프

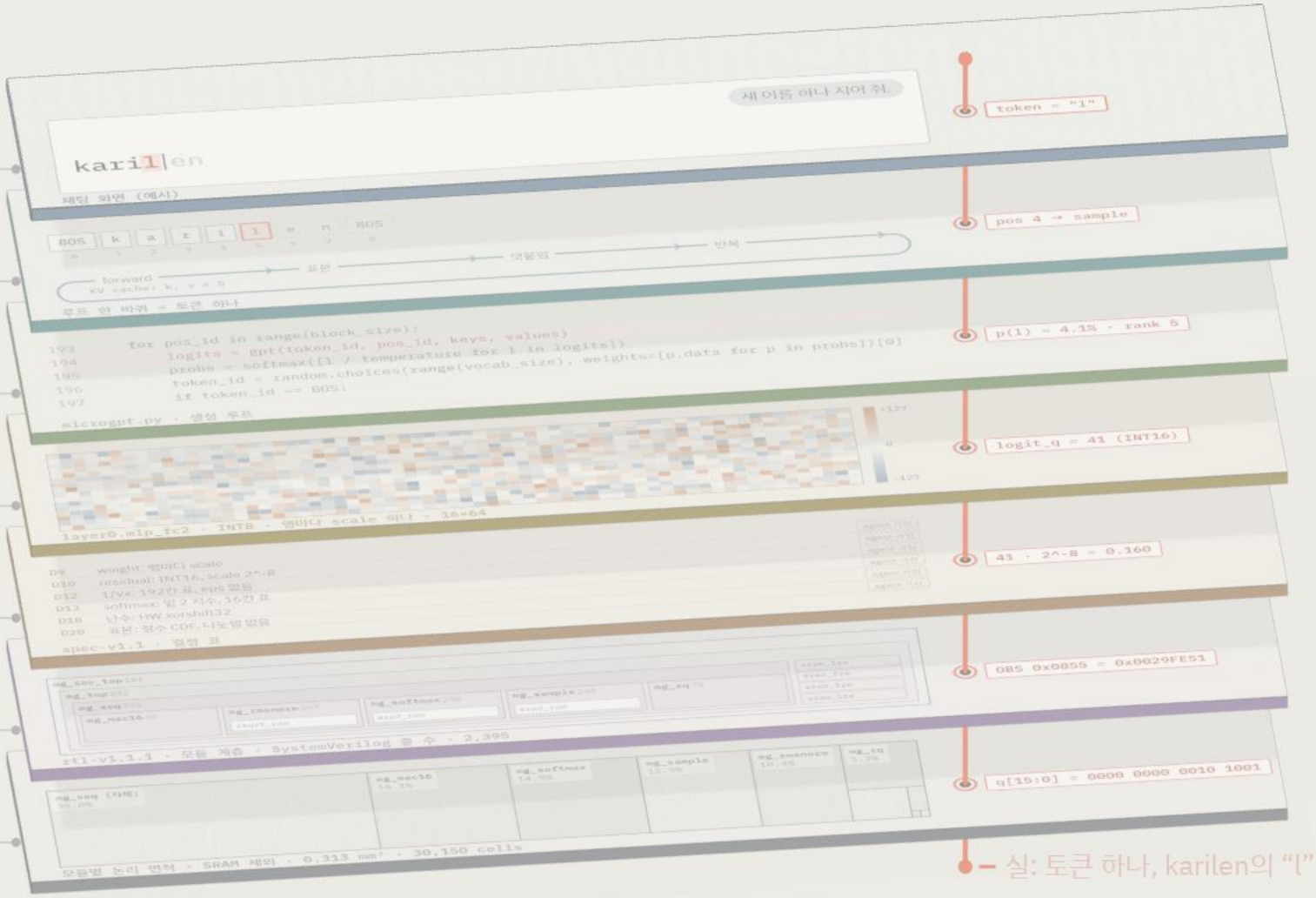
Python

정수 연산

Spec

RTL

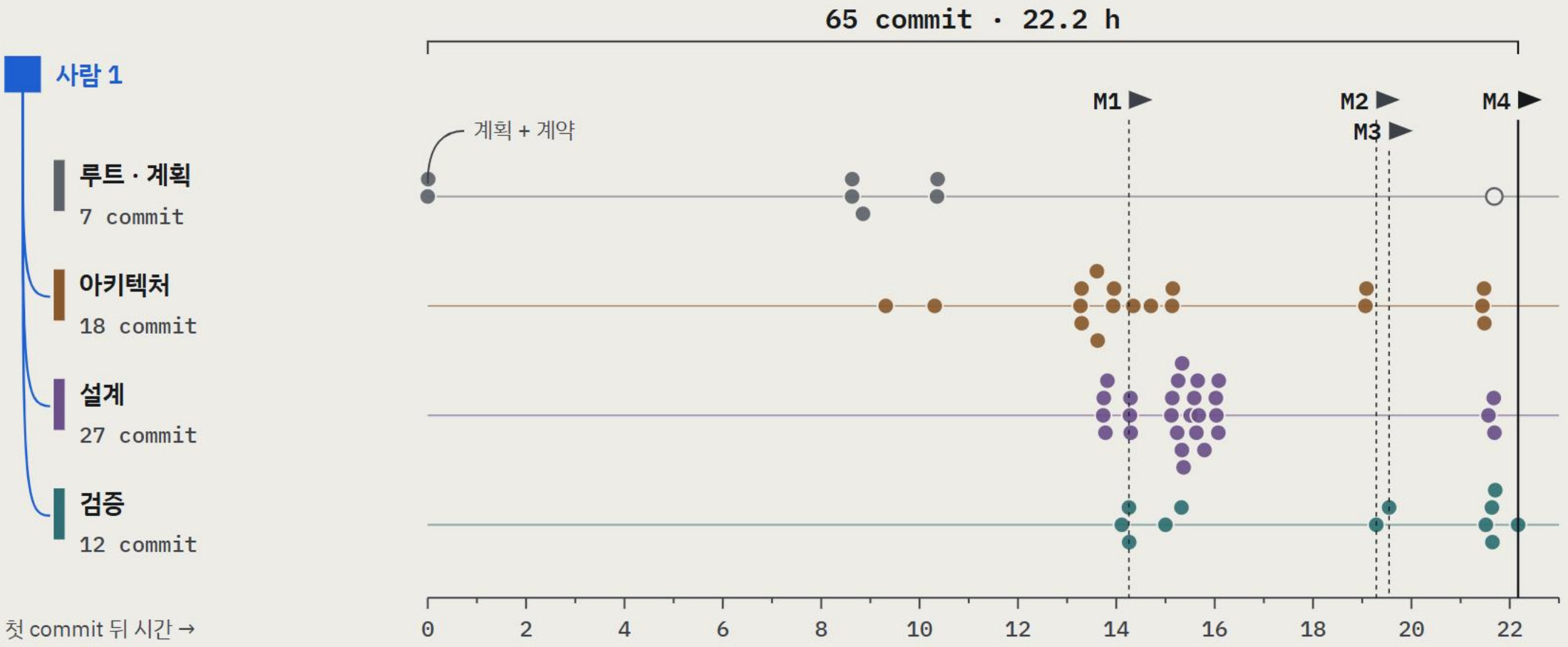
셀



만든 쪽 **agent 팀 셋 + 사람 하나** 검사 시뮬레이터 둘 · **Yosys** 검증한 쪽 **verif (독립)** 사람의 설계 결정 □ **0**

22.2시간에 commit 65개, 계획에서 네 번째 이정표까지

git log · 어떻게 만들었나



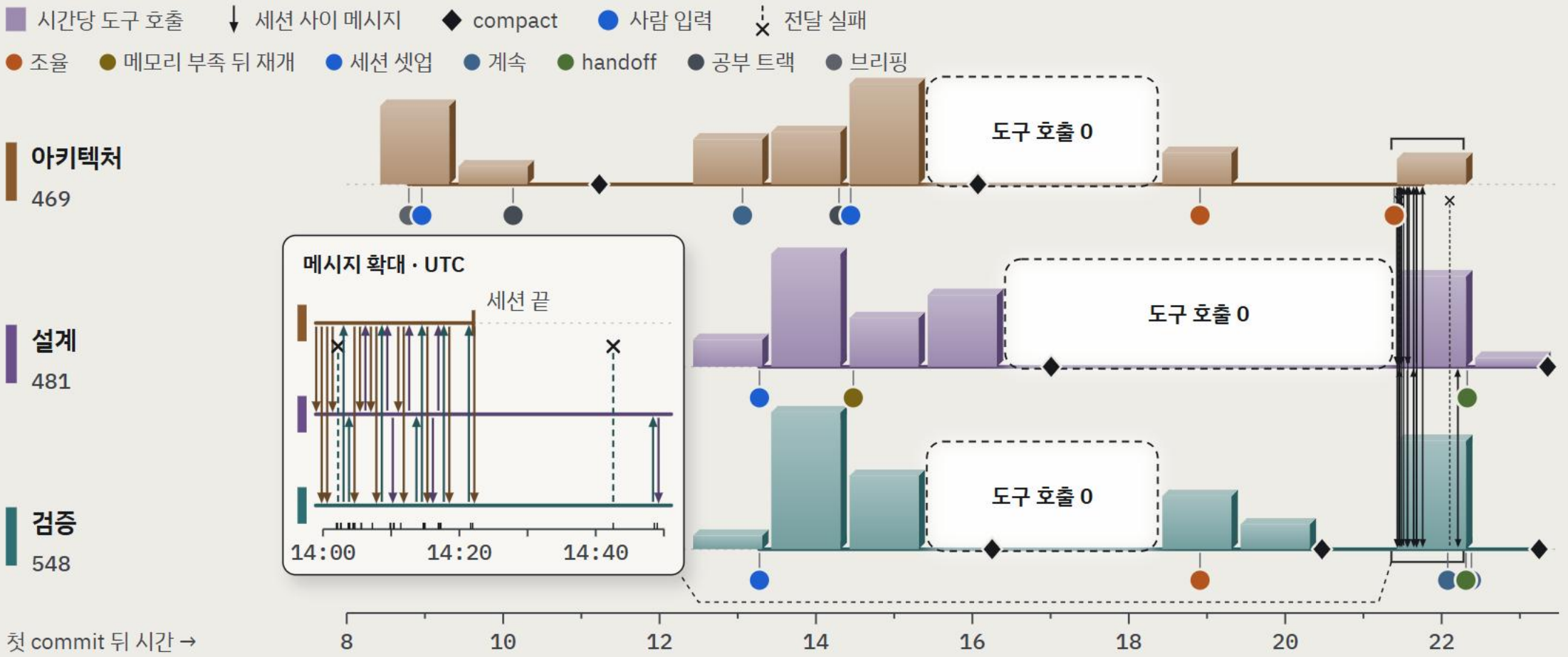
agent 팀 **3** · 사람 **1** · commit **65** · **22.2 h**

계획과 계약에서 시작해 네 번째 이정표의 top 시험 통과까지.

출처 `facts.json process.commits_by_hour, milestone_commits`

세 팀 모두 도구 호출이 0인 시간이 있었다

wf_usage.json · 세션 기록 · 팀마다 한 줄



도구 호출 **1,498** · 세션 사이 메시지 **30 (28 전달, 2 실패) +4** 자기 sub-agent로 · compact **7** · 사람 입력 **16**

막대는 시간당 도구 호출, 마름모는 compact, 점은 사람 입력이다. 세션 사이 메시지 30건은 모두 14:00 UTC 뒤에 있고(확대 창), 2건은 전달되지 않았다.

출처 `wf_usage.json autonomous_branches, human_coding rows`

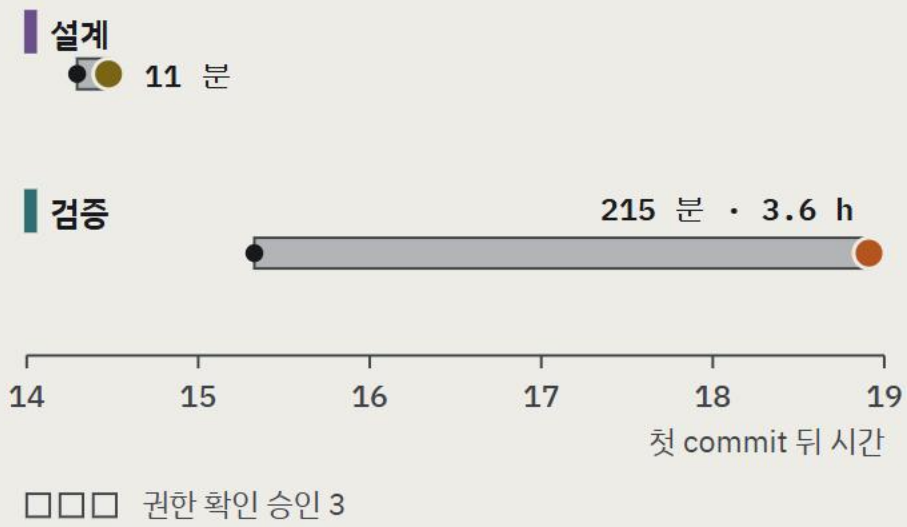
사람 입력 16건 중 설계 결정은 하나도 없었다

wf_usage_human_coding.json · 사람 입력 16건, 종류별

설계 결정	**0**	없음
조율	3	
메모리 부족 뒤 재개	1	
세션 셋업	4	
계속	3	
handoff	2	
공부 트랙	2	
브리핑	1	

운영 개입 4

재개 허락 기다림



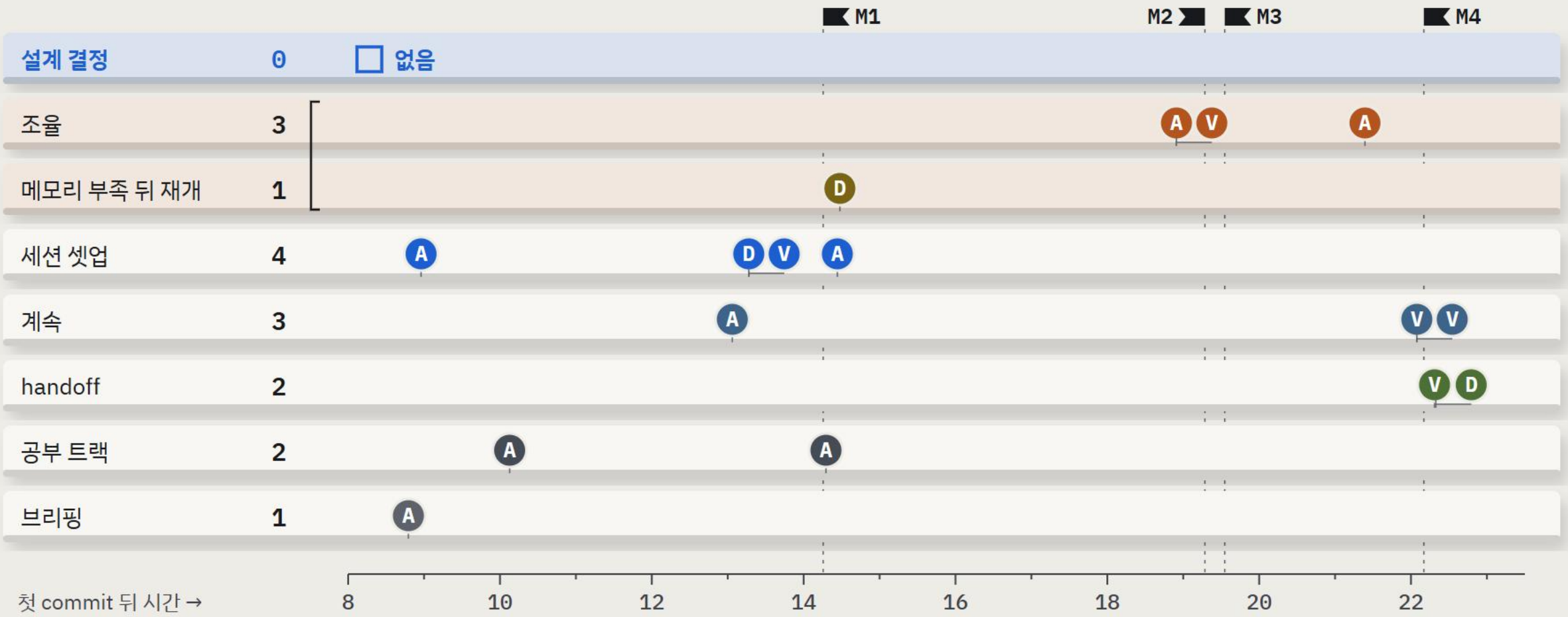
설계 결정 **0** · 운영 개입 **4 (조율 3, 메모리 부족 뒤 재개 1)** · 재개 허락 기다림 **3.6 h**

넷은 운영 개입이었다. 셋은 메시지를 전하거나 기다림을 풀었고, 하나는 메모리 부족 뒤 재실행을 허락했다.

출처 wf_usage_human_coding.json summary

wf_usage_human_coding.json · 사람 입력 16건, 종류와 시각

셋업은 앞에, 조율은 뒤에, handoff는 맨 끝에 왔다



종류는 Claude가 기록에서 분류했고 사람 검토 전이다. 입력 원문은 싣지 않는다

원장엔 사람 개입 1건, 세션 기록엔 4건

자기 보고 vs 기록

원장 · 자기 보고

`*/00_META/ledger.jsonl`

사람 개입

1



sub-agent 종류

`mg-rtl-writer`



×6

≠

≠

세션 기록

`sessions/*.jsonl · 5`

사람 개입

4



운영: 조율 3 · 메모리 부족 뒤 재개 1

sub-agent 종류

`general-purpose`



×6

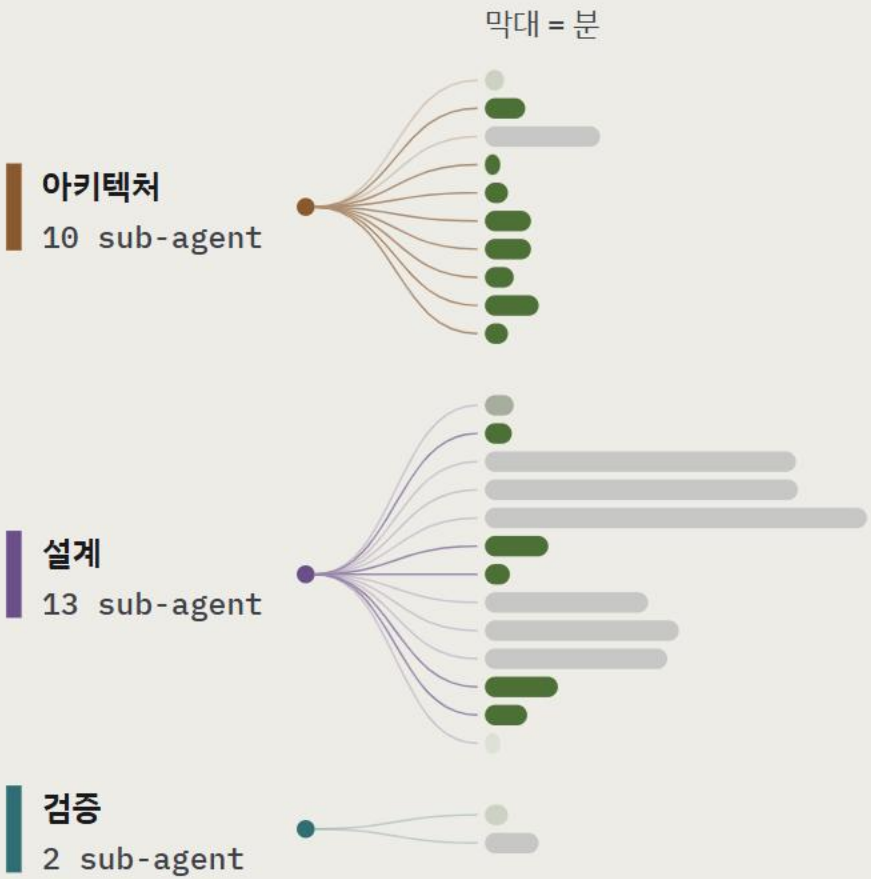
기록에서 센다 **`1 vs 4 · 6 mg-rtl-writer vs 6 general-purpose`**

custom agent 몫으로 적힌 작업 6건은 모두 general-purpose로 돌았다. 자기 보고만 보면 둘 다 숨는다.

출처 `facts.json process.human_interventions_total, human_coding summary`

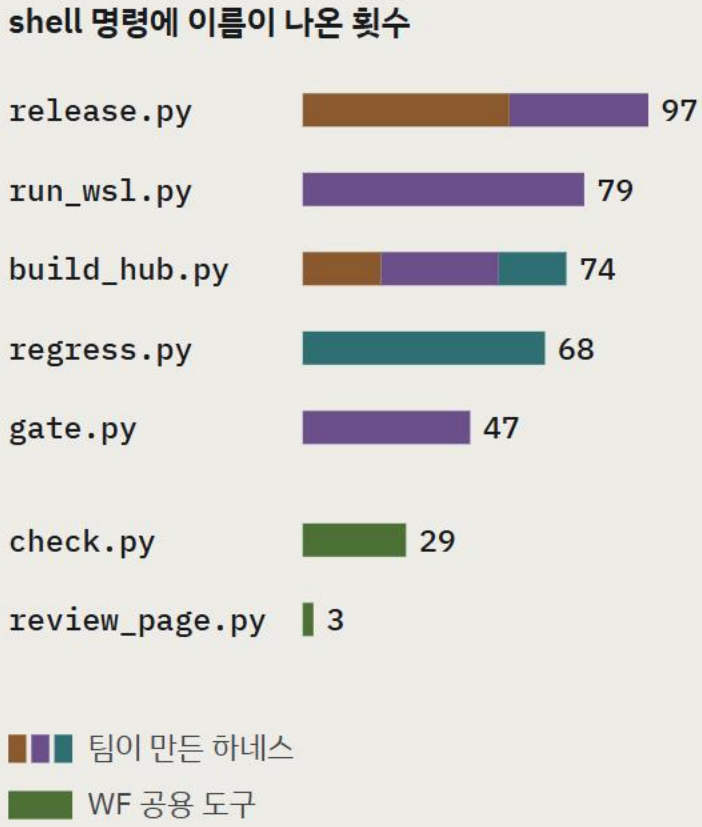
sub-agent 25개 중 13개가 독립 검토였다

wf_usage.json · 누가 일했나



wf-second-look	13
general-purpose	8
wf-project-scout	2
wf-researcher	1
wf-wrap-refiner	1

13 독립 검토



sub-agent **25** · 독립 검토 **13** · shell 명령에 이름이 나온 횟수 **release · run_wsl · build_hub · regress · gate**

shell 명령에 가장 많이 나온 script는 각 팀이 처음에 직접 만든 harness script였다.

출처 wf_usage.json subagents_by_type, scripts_top

검토는 spec에서 가장 많이 찾았고, 질문은 검증이 냈다

WF_IN_PRACTICE_cases.md · 관찰 셋

n = 1 · 한 번의 실행 · 효과가 아닌 관찰

1 검토는 spec에서 가장 많이 잡았다

spec 준비도 검토

M1	M2-rc	M3-rc	v1.1-rc
✓	✓	✓	✓

4/4 지적

RTL release 검토

v0.1.0	v0.2.0	v1.0.0	v1.1.0
0	0	0	0

틀림 0/4

2 spec 초안 질문은 검증에서 나왔다

spec-v0.2-rc · rc 검토

검증	설계
1	0

spec-v1.0-rc · rc 검토

검증	설계
1	0

질문

검증 2 · 설계 0

3 사람은 조율했고, 설계하지 않았다

사람 입력 16



설계 결정 0

- 조율 3
- 메모리 부족 뒤 재개 1

한 번의 실행에서 본 관찰 셋. 사람은 세션을 조율하고 메모리 부족 뒤 일을 다시 시작시켰다.

출처 WF_IN_PRACTICE.md observations, cases C02 C04 C24 C45

한 번, 한 모델 계열: 효과가 아니라 관찰이다

n = 1

한 주제, 한 번의 자율 실행이다.

사람이 결정하는 실행을 다시 하면 비교군이 생긴다.

같은 모델 계열

독립 검토도 모델의 편향은 공유한다.

만들지 않은 맥락이라 독립이지만, 모델은 같다.

분석자 = 실행자 계열

이 분석과 입력 분류도 Claude가 했다.

사람이 원문과 대조해야 주장으로 쓸 수 있다.

근사 분류

shell 명령 · 실패 분류는 정규식이다.

그 수치는 경향으로만 읽는다.

snapshot

세션 기록은 계속 자란다.

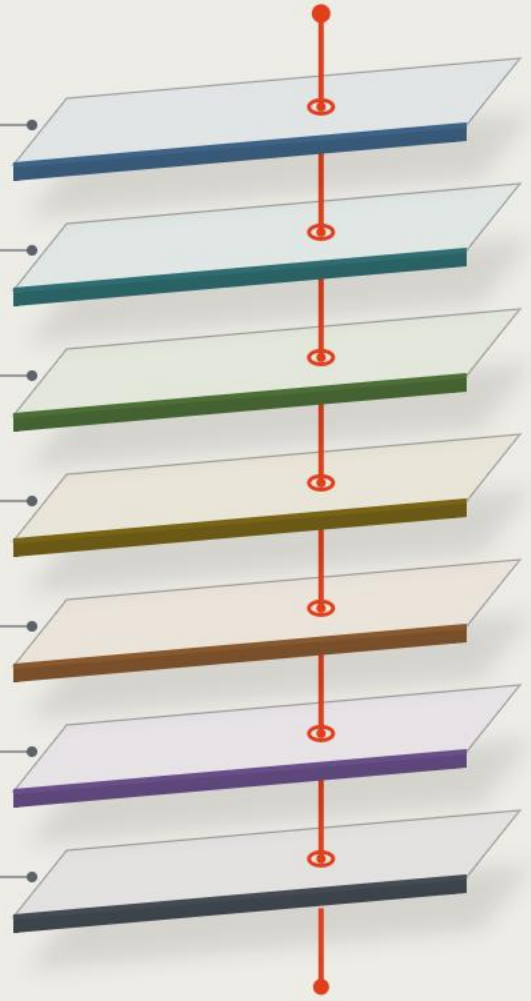
주장은 닫힌 세 팀 세션으로만 한다.



효과가 아니라 관찰

토큰 하나, 층 일곱, 층마다 제 언어

층	무엇인가	이 층에서 본 실
서비스	채팅 API가 토큰을 흘려보낸다	`token = "l"`
루프	forward · 표본 · 덧붙임	`pos 4 → sample`
Python	199줄, 부동소수점	`p(l) = 4.1% · rank 5`
정수 연산	INT8 weight, 고정 scale	`logit_q = 41 (INT16)`
Spec	두 팀이 함께 따르는 계약	`41 · 2^-8 = 0.160`
RTL	SystemVerilog 모듈	`OBS 0x0855 = 0x0029FE51`
셀	sky130 표준 셀	`q[15:0] = 0000 0000 0010 1001`



실: 토큰 하나, karilen의 “l”

출처 scenes/style-frame/data.js thread, the Hero plate tags

토큰 하나는 어디서 오는가?

영상

WF, 이 agent 팀들을 돌린 workflow

youtu.be/l8oLoeUu3UU

다음 실행

사람이 결정한다

